



(51) International Patent Classification:
G06N 3/045 (2023.01)

(21) International Application Number:
PCT/US2025/014152

(22) International Filing Date:
31 January 2025 (31.01.2025)

(25) Filing Language: English

(26) Publication Language: English

(30) Priority Data:
63/548,823 01 February 2024 (01.02.2024) US

(71) Applicants: **DEEPMIND TECHNOLOGIES LIMITED** [GB/GB]; 5 New Street Square, London Greater London EC4A 3TW (GB). **GDM HOLDING LLC** [US/US]; 1600 Amphitheatre Parkway, Mountain View, California 94043 (US).

(72) Inventors: **LIU, Tianlin**; Brandschenkestrasse 110, 8002 Zürich (CH). **GUO, Shangmin**; 30 Rue Montorgueil, 75001 Paris (FR). **CALANDRIELLO, Daniele**; 6 Pancras Square, London Greater London N1C 4AG (GB). **BERTHET, Quentin Didier Olivier**; 8 Rue de Londres, 75009 Paris (FR). **LLINARES LÓPEZ, Felipe**; 8 Rue de Londres, 75009 Paris (FR). **HOFFMANN, Jessica Hélène**;

8 Rue de Londres, 75009 Paris (FR). **DIXON, Lucas Gill**; 8 Rue de Londres, 75009 Paris (FR). **VALKO, Michael**; 8 Rue de Londres, 75009 Paris (FR). **BLONDEL, Mathieu Etienne Gerard**; 8 Rue de Londres, 75009 Paris (FR).

(74) Agent: **PORTNOV, Michael**; P.O. Box 1022, Minneapolis, Minnesota 55440-1022 (US).

(81) Designated States (*unless otherwise indicated, for every kind of national protection available*): AE, AG, AL, AM, AO, AT, AU, AZ, BA, BB, BG, BH, BN, BR, BW, BY, BZ, CA, CH, CL, CN, CO, CR, CU, CV, CZ, DE, DJ, DK, DM, DO, DZ, EC, EE, EG, ES, FI, GB, GD, GE, GH, GM, GT, HN, HR, HU, ID, IL, IN, IQ, IR, IS, IT, JM, JO, JP, KE, KG, KH, KN, KP, KR, KW, KZ, LA, LC, LK, LR, LS, LU, LY, MA, MD, MG, MK, MN, MU, MW, MX, MY, MZ, NA, NG, NI, NO, NZ, OM, PA, PE, PG, PH, PL, PT, QA, RO, RS, RU, RW, SA, SC, SD, SE, SG, SK, SL, ST, SV, SY, TH, TJ, TM, TN, TR, TT, TZ, UA, UG, US, UY, UZ, VC, VN, WS, ZA, ZM, ZW.

(84) Designated States (*unless otherwise indicated, for every kind of regional protection available*): ARIPO (BW, CV, GH, GM, KE, LR, LS, MW, MZ, NA, RW, SC, SD, SL, ST, SZ, TZ, UG, ZM, ZW), Eurasian (AM, AZ, BY, KG, KZ, RU, TJ, TM), European (AL, AT, BE, BG, CH, CY, CZ, DE, DK, EE, ES, FI, FR, GB, GR, HR, HU, IE, IS, IT, LT,

(54) Title: GENERATION OF AN OUTPUT TOKEN SEQUENCE FROM AN INPUT TOKEN SEQUENCE USING TWO LANGUAGE MODEL NEURAL NETWORKS

(57) Abstract: A method of using a first and a second language model neural network to generate an output token sequence from an input token sequence is provided. At least the second network has been fine-tuned. The method comprises, for each position in the output token sequence: (i) generating, using the first network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence, (ii) generating, using the second network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence, (iii) combining, based on a realignment parameter, said scores, and (iv) selecting, for the respective position in the output sequence, a token based on the combined scores.

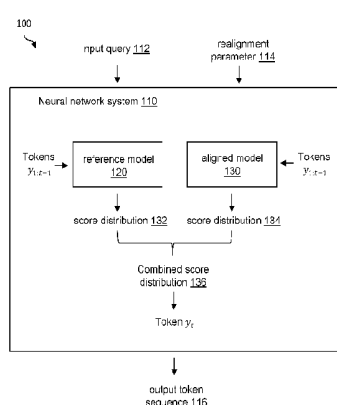


Figure 1



LU, LV, MC, ME, MK, MT, NL, NO, PL, PT, RO, RS, SE,
SI, SK, SM, TR), OAPI (BF, BJ, CF, CG, CI, CM, GA, GN,
GQ, GW, KM, ML, MR, NE, SN, TD, TG).

Published:

— *with international search report (Art. 21(3))*

***GENERATION OF AN OUTPUT TOKEN SEQUENCE FROM AN INPUT TOKEN
SEQUENCE USING TWO LANGUAGE MODEL NEURAL NETWORKS***

CROSS-REFERENCE TO RELATED APPLICATION

[1] This application claims the benefit of U.S. Provisional Application No. 63/548,823 filed February 1, 2024, the disclosure of which is incorporated herein by reference in its entirety.

BACKGROUND

[2] This specification relates to processing data using machine learning models, in particular to the use of language model neural networks to generate an output token sequence from an input token sequence.

[3] Machine learning models receive an input and generate an output, e.g., a predicted output, based on the received input. Some machine learning models are parametric models and generate the output based on the received input and on values of the parameters of the model.

[4] Some machine learning models are deep models that employ multiple layers of models to generate an output for a received input. For example, a deep neural network is a deep machine learning model that includes an output layer and one or more hidden layers that each apply a non-linear transformation to a received input to generate an output.

SUMMARY

[5] It is often desirable to fine-tune pre-trained language model neural networks (LMs) to align the LMs with human preferences to reduce errors and biases in these models. This specification describes a system, implemented as computer programs on one or more computers in one or more locations, and a method that enables control over the degree of alignment (e.g. the strength of a regularization term characterizing the alignment) at “decoding time”, i.e. without requiring retraining multiple models. For example, this allows a user to smoothly transition between unaligned and aligned models, e.g. to identify an optimal regularization strength.

[6] According to a first aspect, there is provided a computer-implemented method of using a first and a second pre-trained language model neural network to generate an output token sequence from an input query. The method comprises autoregressively generating the output token sequence. The term “autoregressive” means that the score distribution generated for any position

in the output token sequence (other than the first position) depends on tokens selected for the earlier positions in the output token sequence. More specifically, for each position in the output token sequence, the first and the second language model neural network process the input token sequence (e.g. independently) to generate a respective score (e.g. logit value) for each token in the output token vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence (of course, for the first position of the output token sequence, no previously selected tokens in the output token sequence are available and thus the scores for the first token are generated based only on the input token sequence). The scores are combined, and each token of the output token sequence is selected based on the combined scores.

[7] The input query may comprise an input token sequence (i.e. a sequence of tokens selected from an input token vocabulary). The input token sequence may comprise “text tokens” from a vocabulary (e.g. tokens representing letters selected from an alphabet (e.g. a natural language alphabet) and/or words or parts of words), “image tokens” (e.g. data representing images or patches of images), and/or “audio tokens” (e.g. data representing sounds, e.g. speech). In particular, the input sequence may represent an interleaved sequence of any one or more of text, image (or video in form of a sequence of images), and/or audio. Similarly, the output token sequence may comprise a sequence of tokens selected from an output token vocabulary (which can be different to the input token vocabulary). The output token sequence may comprise any one or more of text, image and/or audio tokens. Thus, the term “language model neural network” as used herein includes “multimodal” models which are trained not only across text but also across still images, audio (sound signals), and/or videos. That is, either or both of the output or input of the language model neural network can include a media element comprising still image(s), audio, and/or video(s), in addition to the text tokens. The language model neural network may also be capable of processing an input which comprises a media element but not text tokens.

[8] Both language model neural networks are configured to independently process the input token sequence to autoregressively generate, for each position in the output token sequence, a respective score distribution over the vocabulary of output tokens, i.e. a respective score, e.g. a logit value, for each token in a vocabulary of output tokens (the score may be indicative of a predicted likelihood that the corresponding token is the most suited token for the next position in the output token sequence).

[9] Both the first and the second language model neural network may be pre-trained. For example, both language model neural networks may be pre-trained on a large, unlabelled training dataset (e.g. Web data) using unsupervised learning. Both language model neural networks may be pre-trained on the same training dataset. However, in some implementations, the first and the second language model neural network may be pre-trained on different training datasets. In addition to the unsupervised pre-training, the first and the second language model neural network may be further trained using a supervised fine-tuning (SFT) technique, e.g. using datasets that contain expert demonstrations of high-quality, as described below in more detail. Thus, for simplicity, the term “pre-trained” language model neural network is used herein to include language model neural networks that have been SFT-trained after unsupervised learning on a large, unlabelled training.

[10] At least one of the first and the second language model neural network (say the second language model neural network) is further fine-tuned, e.g. trained using alignment training (to steer the network to match human preferences). The alignment training may be characterized by a regularization strength parameter (denoted with the symbol β) that encourages the neural network to stay close to the “unaligned” network (i.e. the pre-trained network without alignment training, also referred to as the “reference” neural network). Thus, the regularization strength parameter β is a hyperparameter that specifies the alignment training process. For example, a large value of the regularization strength parameter β may mean that the trained network will closely follow the reference model, leading to limited alignment. A small value of the regularization strength parameter β may mean that the model will significantly diverge from the reference network causing other performance characteristics to regress. The regularization strength parameter β may be chosen to be the Kullback-Leibler (KL) divergence between the distributions of the unaligned and aligned networks.

[11] It is to be understood that, in some implementations, both the first and the second language model neural network may be further trained by alignment training, as described below in more detail. In these cases, the first and the second language model neural network may be further trained by alignment training with different values for the regularization strength parameter β . However, for simplicity, we assume in the following that the first language model neural network is “unaligned” and that the second language model neural network is “aligned” with a regularization strength parameter β .

[12] In broad terms, the method combines the outputs of the first and the second language model neural networks to emulate the output of an effective language model neural network (referred to as “realigned model”) that has been alignment-trained with an regularization parameter β/λ where λ is a (e.g. user-selected) “realignment parameter”.

[13] More specifically, the method further comprises a step of combining (e.g. additively combining, although in principle the combination may be e.g. multiplicative), based on the realignment parameter λ , the scores generated by the first and second language model neural networks. The value of the realignment parameter λ can be selected at decoding-time, i.e. after training of the first and the second language model neural networks. The realignment parameter λ specifies the degree of alignment of the “realigned model”, i.e. the degree of alignment on which the output token sequence is to be based.

[14] The method further comprises selecting the next token in the output token sequence from the vocabulary of output tokens based on the combined scores, e.g. by applying a softmax function to the combined scores and the next token in the output token sequence may be selected (drawn) based on the output of the softmax function (for example, the token having a highest score under the score distribution may be selected as the token for the respective position in the output token sequence). The aforementioned steps are repeated until the end of the output token sequence is reached.

[15] The above-described method enables, for example, efficient testing of various regularization strengths without retraining, thereby saving computational cost at training time. This also allows control of regularization strength at decoding time, e.g. depending on the user or the task.

[16] The above-described method may be used to find an optimized value of the regularization strength parameter β (e.g. an value that represents the best reward-regularization trade-off) without having to train a large number of models with distinct regularization strengths.

[17] More specifically, according to a second aspect, there is provided a computer-implemented method of using a first and a second language model neural network for determining an optimized value of a regularization strength parameter characterizing a fine-tuning step. The first and the second language model neural network are configured to process an input token sequence to generate, for each position in an output token sequence, a respective score for each token in a vocabulary of output tokens. At least the second language model neural network has been fine-

tuned, the fine-tuning being characterized by a corresponding regularization strength parameter having a first value.

[18] The method comprises obtaining a plurality of candidate values of a realignment parameter. For each candidate value of the realignment parameter, an output token sequence is generated from an input token sequence by, for each position in the output token sequence: generating, using the first language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence; generating, using the second language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence; combining, based on said realignment parameter, said scores generated by the first and second language model neural networks; and selecting, for the respective position in the output sequence, a token from the vocabulary of output tokens based on the combined scores.

[19] The method further comprises evaluating the plurality of generated output token sequences to select a value for the realignment parameter and determining the optimized value of a regularization strength parameter based on the selected value of the realignment parameter. In some implementations, at least 1000 input queries (i.e. input token sequences) are processed for each candidate value of the realignment parameter λ (the same set of input queries may be processed for each candidate value of the realignment parameter λ). The resulting output token sequences may be evaluated to select the “best” value λ_{best} of the realignment parameter. From the selected value of the realignment parameter λ , the optimized value β_{best} for the regularization strength parameter may be determined, for example, as $\beta_{\text{best}} = \beta / \lambda_{\text{best}}$.

[20] Particular embodiments of the subject matter described in this specification can be implemented so as to realize one or more of the following advantages.

[21] Selecting an appropriate level of regularization is important for the performance of an aligned LM. Insufficient regularization can lead to reduced model capabilities, for example, due to reward hacking, whereas excessive regularization hinders alignment. The system described in this specification enables the exploration and evaluation of different regularization strengths in aligned models. This has been confirmed experimentally in length-reward task and in summarization task examples as described below in more detail. Advantageously, the system permits a reduction of the computational cost at training time because it does not require retraining multiple models with varying regularization strength. It is also advantageous that the system

described in this specification can be easily implemented. This is because general-purpose open-source models (e.g. Llama-2 or Mistral 7B) may be released as base and instruction-fine-tuned checkpoints. These models can respectively serve as reference and aligned model in the described system. By trading off between the base and aligned models, practitioners can therefore easily adapt language generation to specific user preferences without the need for extensive retraining.

[22] In some implementations, once the optimized value β_{best} for the regularization strength parameter has been determined, a corresponding model may be trained, i.e. the method may further comprise steps of performing pre-training of a target language model neural network, and performing alignment training of the pre-trained target language model neural network, where the alignment training is characterized by a corresponding regularization strength parameter having the optimized value β_{best} . The target language model neural network may have the same architecture as the first and/or the second language model neural network. It is advantageous that the system described in this specification can be used to identify the promising regularization strengths and then retrain the model only at these values. This reduces the overall training cost and does not incur a computational overhead at decoding time.

[23] The details of one or more embodiments of the subject matter of this specification are set forth in the accompanying drawings and the description below. Other features, aspects, and advantages of the subject matter will become apparent from the description, the drawings, and the claims.

BRIEF DESCRIPTION OF THE DRAWINGS

[24] FIG. 1 shows an example computer system that processes an input token sequence to generate an output token sequence.

[25] FIG. 2 is a flow diagram of an example process of generating an output token sequence.

[26] FIG. 3 shows an example computer system that determines an optimized value of a regularization strength parameter.

[27] FIG. 4 is a flow diagram of an example process for determining an optimized value of a regularization strength parameter.

[28] FIGS. 5 to 12 illustrate experimental results from example implementations of the computer systems of FIG. 1.

[29] Like reference numbers and designations in the various drawings indicate like elements.

DETAILED DESCRIPTION

[30] Language model neural networks (LMs) are neural networks which process an input query, comprising a sequence of tokens and/or other data such as a media element, to generate an output, comprising an output sequence of tokens typically representing a piece of text which is a sensible response to the input query, e.g. an answer to a question posed by the input query. More specifically, a LM, conditioned on a query sequence $x = (x_1, \dots, x_m) \in \mathcal{X}$, parametrizes a probability distribution over response sequences $y = (y_1, \dots, y_n) \in \mathcal{Y}$. The probability $\pi(y|x)$ can be factorized using the chain rule of probability: $\pi(y|x) = \pi(y_1|x)\pi(y_2|y_1, x) \dots \pi(y_n|y_1, \dots, y_{n-1}, x)$. This factorization is beneficial since the log-probability $\log \pi(y|x)$ is easy to compute, enabling maximum likelihood (MLE) based training. Further, it is easy to generate independent and identically distributed (i.i.d.) samples $y \sim \pi(\cdot | x)$ at decoding time.

[31] Sometimes, a LM is first pre-trained on a large, unlabeled (text) dataset (e.g. Web data) and then fine-tuned for downstream tasks. The pre-training may be performed by self-supervised learning (also known as predictive learning) in which the LM learns to correctly predict continuations of received samples of a text database, such as a large, publically-available database of natural language.

[32] Fine-tuning is performed because self-supervised LMs often exhibit factual errors, biases, and other undesirable behaviour. Thus, the pre-trained LM may be fine-tuned to align with human expectations and values. For example, it can be desirable (depending on the intended use of the LM) to steer the pre-trained LM towards responses that are harmless, helpful, engaging, enjoyable, etc. To this end, additional training, known as supervised fine-tuning (SFT), may be performed. For example, SFT may involve supervised training on smaller, more carefully curated datasets that contain expert demonstrations of high-quality responses. These datasets showcase high-quality responses, highlighting desired behaviours like following instructions, engaging in dialogue, summarization or the like. In this case, the fine-tuned model can be obtained through maximum likelihood estimation.

[33] Alternatively or additionally, SFT can also be performed with a human-annotated preference dataset. Such human-annotated preference datasets comprise human assessments of preferred and unpreferred responses (such a pairwise-preference dataset is more affordably and abundantly collected than the aforementioned expert-generated response demonstrations). This

process of using a preference dataset for SFT of a pre-trained LM is also known as “alignment training”. Alignment training can be characterized by a hyperparameter β (“regularization strength parameter”) that controls the deviation of the aligned model from the unaligned model (i.e. the pre-trained model without alignment training, also referred to as the “reference” model).

[34] In general, many possibilities of performing alignment training exists such as reinforcement learning from human feedback (RLHF) and direct preference learning (DPL) methods (e.g. direct preference optimisation (DPO), sequence likelihood calibration with human feedback (SLiC), and identity policy optimisation). These alignment training techniques are typically cast as optimizing a tradeoff between human preference rewards and a proximity regularization term that encourages staying close to the unaligned model.

[35] More specifically, reinforcement learning from human feedback (RLHF) performs alignment by first learning a scalar-valued reward model, that mimics human judgment, and then employs reinforcement learning to optimize the LM against this reward. The scalar-valued reward function (denoted $r : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}$) captures the affinity between an input query x and a response token sequence y . Since hand-crafting a reward function is usually not easy, the reward function r may be learned from pairwise human preferences. Once the reward is learned, the aligned LM can be expressed using a regularised alignment objective based on the expected reward and the Kullback-Leibler (KL) divergence between the distributions of the unaligned and aligned models. Thus, aligning the LM may be cast as a trade-off between maximizing the expected reward and staying close, in the KL sense, to the distribution obtained after pre-training:

$$\pi^*(\beta) := \operatorname{argmax}_{\pi} \left\{ \mathbb{E}_{x \sim p_x} \mathbb{E}_{y \sim \pi(\cdot|x)} r(x, y) - \beta \mathbb{E}_{x \sim p_x} \text{KL}(\pi(\cdot|x) \parallel \pi^{\text{ref}}(\cdot|x)) \right\} \quad (1)$$

where $\pi^*(\beta)$, $\pi^{\text{ref}}(\cdot|x)$ are the probability functions of the aligned and unaligned model respectively, and p_x is the distribution of queries. Assuming that the LM is a parametrized autoregressive model π_{θ} (such as a transformer network), Equation (1) (which is a maximization problem is over the space of distributions) can be cast into a maximization over the space of parameters θ :

$$\operatorname{argmax}_{\theta} \left\{ \mathbb{E}_{x \sim p_x} \mathbb{E}_{y \sim \pi_{\theta}(\cdot|x)} r(x, y) - \beta \mathbb{E}_{x \sim p_x} \text{KL}(\pi_{\theta}(\cdot|x) \parallel \pi^{\text{ref}}(\cdot|x)) \right\} \quad (2).$$

An approximate model π_θ can be obtained based on Equation (2) by using reinforcement learning algorithms such as policy gradient optimizations methods (e.g. proximal policy optimization (PPO)).

[36] In Equations (1) and (2), the regularization strength parameter β controls a balance between the expected reward and the KL divergence. For example, a large value of the regularization strength parameter β means that the aligned LM will closely follow the reference model, leading to limited alignment. A small value of the regularization strength parameter β means that the LM will significantly diverge from the reference network. Selecting a favourable trade-off between reward and KL divergence is important for the importance of the resulting model. When the regularization strength parameter β is chosen too large, the strong KL regularization encourages the aligned model to closely follow the reference model, limiting the effectiveness of alignment. If the regularization strength parameter β is chosen too small, the aligned model will typically significantly deviate from the reference model. This can cause reward hacking where the aligned model overfits the reward compromising important abilities like coherence and topicality learned during pre-training or SFT. For example, allowing the aligned model to strongly deviate from the reference model, as measured by increased KL divergence, leads to higher rewards but at the cost of generating less natural samples. In summarization tasks, allowing the model to increase its reward can initially enhance the summarization quality, although allowing the reward to become too high causes the summarization quality to deteriorate as the aligned model overfits. Thus, selecting an appropriate level of regularization is important. Conventional methods for finding the optimal regularization level require retraining multiple models with varying regularization strength. This process, however, is resource-intensive, especially for large models.

[37] FIG. 1 shows an example computer system 100. The computer system 100 is an example of a system, implemented as computer programs on one or more computers in one or more locations, in which the systems, components, and techniques described below are implemented. In broad terms, the computer system 100 combines the outputs of an unaligned LM and an aligned LM to emulate the output of an effective LM having a degree of alignment that can be selected at decoding-time (i.e. after training of the LMs).

[38] More specifically, the computer system 100 comprises a neural network system 110 configured to process an input query 112 and a realignment parameter 114 to generate an output token sequence 116. The input query 112 defines a query or task for the neural network system

110, and may be obtained from a user (e.g. via user interface), from another software application, or the like. The input query 112 can include one or more modalities including text, image, audio, video, or a combination of one or more such modalities. In general, the network input can include multiple modalities depending on the specific architecture of the neural network system.

[39] The input query 112 comprises an input token sequence (i.e. a sequence of tokens selected from an input token vocabulary) denoted $x = (x_1, \dots, x_m) \in \mathcal{X}$. The tokens can be “text tokens” from a vocabulary (e.g. tokens representing letters selected from an alphabet (e.g. a natural language alphabet) and/or words or parts of words), “image tokens” (e.g. data representing images or patches of images), and/or “audio tokens” (e.g. data representing sounds, e.g. speech). In particular, the input sequence may represent an interleaved sequence of text, image (or video in form of a sequence of images), and/or audio.

[40] As described below in more detail, the realignment parameter 114, denoted with the symbol λ , specifies the degree of alignment on which the output token sequence 116 is to be based. Similar to the input query 112, the realignment parameter 114 may be obtained from the user (e.g. via user interface).

[41] The output token sequence 116, denoted $y = (y_1, \dots, y_n) \in \mathcal{Y}$, represents a response to the input query 112 and comprises a sequence of tokens selected from an output token vocabulary (which can be different to the input token vocabulary). Similar to the input token sequence, the output token sequence 116 may comprise text, image and/or audio tokens.

[42] The neural network system 110 comprises a trained reference model 120 (“first language model neural network”) and a trained aligned model 130 (“second language model neural network”). The models 120, 130 have respective pluralities of learnable/tunable numerical parameters (e.g. at least 1000, at least 100,000, or more numerical parameters). The models 120, 130 have been trained (i.e. the numerical values of the respective parameters have been adjusted) based on training data as described further below.

[43] In general, the reference model 120 and the aligned model 130 may be implemented using known language model neural network architectures. For example, the models 120, 130 may comprise a transformer network, i.e. a neural network subsystem including one or more transformer blocks or self-attention layers. A transformer block typically includes an attention or self-attention neural network layer followed by a feedforward neural network. An attention, or self-attention, neural network layer is a neural network layer that includes an attention or self-

attention mechanism that operates over an attention layer input to generate an attention layer output. More specifically, the models 120, 130 may each comprise a respective transformer neural network with an architecture similar to the transformer neural network described in Vaswani et al, Advances in neural information processing systems, 30, 2017. For example, the models 120, 130 may comprise an architecture similar to a (standard) vision transformer neural network (e.g. similar to the Vision Transformer (ViT) described in Dosovitskiy et al, arXiv:2010.11929). Thus, in some implementations, the models 120, 130 are vision language models which can process both visual (image and video) data along with text data as the input query.

[44] In the embodiment of Figure 1, the reference model 120 and the aligned model 130 have the same network architecture. In other implementations, the reference model and the aligned model may have the different network architectures.

[45] The reference model 120 is configured to process the input token sequence x of the input query 112 to autoregressively generate, for each position in the output token sequence y , a respective score distribution 132 over the vocabulary of output tokens, i.e. a respective score for each token in a vocabulary of output tokens. The score may be indicative of a predicted likelihood that the corresponding token is the most suited token for the next position in the output token sequence. More specifically, in the embodiment of Figure 1, the reference model 120 is configured to generate, for each position in the output token sequence y , the score distribution 132 so as to comprise a logit for each token in the vocabulary. Denoting the reference model 120 with $f^{ref}(\cdot)$ (where the symbol “.” represents the model input), the output of reference model 120 for position t of the output token sequence y (with $1 \leq t \leq n$) can be represented as a vector $h_t^{ref} := f^{ref}(x, y_1, \dots, y_{t-1})$ (with $h_t^{ref} \in \mathbb{R}^V$, where V is the output token vocabulary size) that comprises a corresponding component for each logit generated by the reference model 120 for position t . By applying the softmax function to the vector h_t^{ref} , a next-token probability distribution π^{ref} for the reference model 120 can be obtained as $\pi^{ref}(\cdot | x, y_1, \dots, y_{t-1}) = \text{softmax}(h_t^{ref})$ (where the symbol “.” represents the possible next tokens). As described below in detail, it is not necessary to explicitly generate the next-token probability distribution π^{ref} in order to obtain the output sequence 116.

[46] The reference model 120 has been trained using training data. More specifically, the reference model 120 is normally been pre-trained on a large, unlabelled training dataset (e.g. Web

data) using unsupervised learning, and preferably further trained using a supervised fine-tuning (SFT) technique, e.g. using datasets that contain expert demonstrations of high-quality. In the embodiment of Figure 1, the reference model 120 has not been further fine-tuned using “alignment training” (i.e. based on a preference dataset for SFT). Thus, the reference model 120 may be considered to be “unaligned”.

[47] The aligned model 130 is generally similar to the reference model 120 except that the aligned model 130 has been fine-tuned using alignment training characterized by the regularization strength parameter β . More specifically, the aligned model 130 is configured to process the input token sequence x of the input query 112 to autoregressively generate, for each position in the output token sequence y , a respective score distribution 134 over the vocabulary of output tokens, i.e. a respective score for each token in a vocabulary of output tokens. As for the reference model 120, in the embodiment of Figure 1, the aligned model 130 is configured to generate, for each position in the output token sequence y , the score distribution 134 so as to comprise a logit for each token in the vocabulary. Denoting the aligned model 130 with $f_{\beta}^{\theta}(\cdot)$, the output of aligned model 130 for position t of the output token sequence y (with $1 \leq t \leq n$) can be represented as a vector $h_t^{\theta}(\beta) := f_{\beta}^{\theta}(x, y_1, \dots, y_{t-1})$ (with $h_t^{\theta}(\beta) \in \mathbb{R}^V$) that comprises a corresponding component for each logit generated by the aligned model 130 for position t . By applying the softmax function to the vector $h_t^{\theta}(\beta)$, a next-token probability distribution π_{θ} for the aligned model 130 can be obtained as $\pi_{\theta}(\beta)(\cdot | x, y_1, \dots, y_{t-1}) = \text{softmax}(h_t^{\theta}(\beta))$. As for the reference model 120, it is not necessary to explicitly generate the next-token probability distribution $\pi_{\theta}(\beta)$ in order to obtain the output sequence 116.

[48] Similar to the reference model 120, the aligned model 130 has been trained using training data. More specifically, the aligned model 130 may have been pre-trained, and preferably further trained using a supervised fine-tuning (SFT) technique. In the embodiment of Figure 1, the reference model 120 and the aligned model 130 have been trained using the same training data (however this is not a strict requirement, i.e. in other implementations, the reference model and the aligned model may be trained on different training data). In addition to the pre-training and the optional SFT training, the aligned model 130 has been further fine-tuned using alignment training (i.e. using pairwise preference training data) to steer the aligned model 130 to match human preferences. As described above, the alignment training is characterized by the regularization

strength parameter β that controls the deviation of the aligned model 130 from the model without alignment training i.e. the reference model 120. The alignment training may be implemented using the aforementioned possibilities such as reinforcement learning from human feedback (RLHF) and direct preference learning (DPL) methods. For example, the alignment training of the aligned model 130 may comprise comparing output token sequences based on scores generated by the aligned model 130 upon receiving corresponding sample input token sequences, with corresponding target output token sequences for the sample input token sequences.

[49] The neural network system 110 computes, from the scores generated by the models 120, 130 and the realignment parameter λ , a per-token probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ of a “realigned” model that has been alignment-trained with an regularization parameter β/λ . More specifically, the per-token probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ of the realigned model may be expressed as

$$\hat{\pi}_\theta(\beta/\lambda)(y_t|x, y_{1:t-1}) := \frac{1}{Z(x, y_{1:t-1})} \pi^{\text{ref}}(y_t|x, y_{1:t-1}) \left[\frac{\pi_\theta(\beta)(y_t|x, y_{1:t-1})}{\pi^{\text{ref}}(y_t|x, y_{1:t-1})} \right]^\lambda \quad (3)$$

where $Z(x, y_{1:t-1}) := \sum_{y_t} \pi^{\text{ref}}(y_t|x, y_{1:t-1}) \left[\frac{\pi_\theta(\beta)(y_t|x, y_{1:t-1})}{\pi^{\text{ref}}(y_t|x, y_{1:t-1})} \right]^\lambda$ is the normalization constant.

Equation (3) allows generating the output sequence of the realigned model one token at a time based on the per-token probability distributions π^{ref} and $\pi_\theta(\beta/\lambda)$ and the realignment parameter λ . However, the computation of the per-token probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ of Equation (3) can be simplified using the fact that the geometric mean is equivalent to the arithmetic mean in log scale. More specifically, the realigned model $\hat{\pi}_\theta(\beta/\lambda)$ can be equivalently written as

$$\hat{\pi}_\theta(\beta/\lambda)(\cdot|x, y_{1:t-1}) = \text{softmax}(\lambda h_t^\theta(\beta) + (1 - \lambda) h_t^{\text{ref}}) \quad (4)$$

where h_t^{ref} and $h_t^\theta(\beta)$ denote the above described vectors comprising the respective logits generated by the reference model 120 and the aligned model 130.

[50] The term $h_t^\theta(\beta) + (1 - \lambda) h_t^{\text{ref}}$ in Equation (4) linearly combines the reference logits h_t^{ref} and the aligned logits h_t^θ . In the special case of $\lambda = 0$, the regularization strength β/λ is infinite, and $\hat{\pi}_\theta(\beta/\lambda)$ corresponds to the per-token distribution of the reference model 120. When $\lambda = 1$, the regularization strength β/λ is 1, and $\hat{\pi}_\theta(\beta/\lambda)$ corresponds to the per-token distribution of the aligned model 130. Thus, the configurable realignment parameter λ provides the flexibility of exploring different reward-regularization trade-offs. The realignment parameter λ is not bounded above by 1. When $\lambda > 1$, the realigned model uses a smaller regularization strength β/λ than the

base KL strength β . Thus, in some implementations, the value of λ may be selected to be larger than zero and different from (e.g. lower than) one.

[51] Thus, the neural network system 110 is further configured to combine, for each position in the output sequence 116, the scores 132, 134 generated by the reference model 120 and the aligned model 130 based on the realignment parameter 114, and to select a token from the vocabulary of output tokens based on the combined scores 136. In this way, the neural network system 110 can sequentially select the tokens of the output sequence 116 until the end of the output token sequence is reached.

[52] More specifically, for each position in the output token sequence 116, the scores may additively be combined to satisfy the expression $\lambda h_t^\theta(\beta) + (1 - \lambda) h_t^{\text{ref}}$. To select a token, the neural network system 110 may be configured to apply a softmax function to the combined scores (e.g. implement Equation (4)) and select (draw) the token for a respective position in the output token sequence based on the output of the softmax function (for example, the token having a highest score under the score distribution may be selected as the token for the respective position in the output token sequence). Applying the softmax function to the combined scores maps the combined scores onto to a probability distribution from which the token can be selected. Alternatively, the neural network system 110 may be configured to, for each position in the output token sequence 116, multiplicatively combine the scores based on the realignment parameter 114, and select the token based on the multiplicatively combined scores. For example, the neural network system 110 may multiplicatively combine the scores based on the realignment parameter 114 to compute the per-token probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ of the realigned model as defined in Equation (3), and select a token based on the computed probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ (for example, the token having the highest probability under the probability distribution $\hat{\pi}_\theta(\beta/\lambda)$ may be selected as the token for the respective position in the output token sequence).

[53] Thus, the computer system 100 allows control of regularization strength at decoding time, e.g. depending on the user or the task. The computer system 100 can be used efficiently test various regularization strengths without retraining, thereby saving computational cost at training time. As described below in more detail, experiments were performed which demonstrate that the proposed approach of combining scores (logits) from two models leads to a higher performance than an alternative approach of combining the model weights (i.e. the values of the model parameters).

[54] While the neural network system of Figure 1 comprises an “unaligned” reference model 120 (i.e. a model that has not been trained using alignment training), in other implementations, the neural network system may comprise, instead of the unaligned reference model, a further aligned model that is aligned to a different degree compared to the aligned model 130 (i.e. $\beta_1 \neq \beta_2$ where β_1 , β_2 respectively denote the regularization strength parameters of the models).

[55] FIG. 2 is a flow diagram of an example process 200 of using a neural network system configured to process an input query comprising an input token sequence to generate an output sequence. The process 200 of FIG. 2 may be implemented by one or more computers in one or more locations. For example, the process 200 may be implemented by the system of FIG. 1, and for convenience the process is described with reference to FIG. 1.

[56] At an initial step 202, the input query 112 is obtained based on input from a user (e.g. via a user interface) or from a software application in communication with the neural network system 100. An interface may be provided between (i) the user or the software application, and (ii) the neural network system to generate input sequence tokens of the input query 112 from the input from the user or software application (e.g. if the input is a string of characters, raw image data (e.g. an image captured by a camera) or audio data (e.g. audio data captured by a microphone). For example, the interface may include an embedding system for converting characters (e.g. typed by a user) into text tokens, a set of pixel values defining the image into image tokens, and/or frequency/amplitude data into audio tokens.

[57] Further, a value for the realignment parameter 114 is obtained. The realignment parameter 114 may be obtained from the user or the software application or may have been stored on a storage device of the one or more computers implementing the process 200.

[58] The process 200 involves combining the outputs of an unaligned reference LM and an aligned LM (having an alignment characterised by the regularization strength parameter β) to emulate the output of an effective LM having a degree of alignment that depends on the realignment parameter 114 and the regularization strength parameter β . To this end, at step 204, for each position in the output token sequence, the reference model 120 generates a respective score (e.g. logit value) for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence. Similarly, at step 206, the aligned model 130 generates a respective score (e.g. logit value) for each token in the vocabulary

based on the input token sequence and based on any previously selected tokens in the output token sequence.

[59] At step 208, the scores generated by the reference model 120 and the aligned model 130 are combined (e.g. additively or multiplicatively) based on the realignment parameter 114.

[60] At step 210, a token from the vocabulary of output tokens is selected based on the combined scores. This can involve mapping (e.g. by applying the softmax function) the combined scores onto to a probability distribution and selecting the token based on the probability distribution. The combining of the scores and the mapping of the combined scores onto the probability distribution may be implemented as one computational step, e.g. the generated scores and the realignment parameter 114 may be used to evaluate Equations (3) or (4) to the per-token probability distribution $\hat{\pi}_{\theta}(\beta/\lambda)$.

[61] Steps 204 to 210 are repeated until the end of the output token sequence 116 is reached. This may for example be because step 210 generates an output token which indicates that the process 200 should terminate.

[62] The output token sequence 116 may be transmitted to a user. For example, if the output tokens are text tokens, text based on the text tokens may be displayed to a user. Alternatively or additionally, the output tokens may be transmitted to a system which converts them into image data (pixel-level intensity data) which may be displayed to a user and/or audio data (sound amplitude or frequency data) which transmitted to a speaker to generate sound based on the audio tokens.

[63] FIG. 3 shows a variation of the computer system 100 of FIG. 1. The computer system 300 of FIG. 3 is configured to determine an optimized value of the regularization strength parameter that characterizes the fine-tuning (alignment) of a LM. In broad terms, the computer system 300 enables sweeping over a plurality of values of the regularization strength which streamlines the hyperparameter selection and reduces the number of retraining runs needed to get optimal regularization strengths.

[64] The computer system 300 of FIG. 3 is identical to the computer system 100 of FIG. 1 except that the computer system 300 further comprises a plurality of input queries 312, a plurality of candidate values of the realignment parameter 314 and an evaluation unit 320. In some implementations, the plurality of input queries 312 comprises at least 1000 input queries (i.e. input token sequences).

[65] The computer system 300 is configured to determine, using the neural network system 110, a plurality of output token sequences 316 comprising a respective output token sequence for each input query in the plurality of the input queries 312 and for each candidate value in the plurality of candidate values of the realignment parameter 314.

[66] The evaluation unit 320 is configured to evaluate the plurality of generated output token sequences 316 and the computer system 300 is further configured to select a value of the realignment parameter, denoted λ_{best} , based on the evaluations. More specifically, the evaluation unit 320 may assign a respective evaluation score to each candidate value of the realignment parameter based on the generated output token sequences 316. In these cases, the computer system 300 may be configured to select, as λ_{best} , the value of the candidate values of the realignment parameter that corresponds to the highest evaluation score.

[67] The evaluation unit 320 may be implemented as a further LM configured to act as a “judge” for the generated output token sequences 316. In these cases, the LM of the evaluation unit 320 is preferably more “capable” than the reference model 120 and the aligned model 130. For example, the LM of the evaluation unit 320 may have a larger number of learnable parameters compared to the reference model 120 and the aligned model 130 (e.g. at least 10 times more learnable parameters), and/or the LM of the evaluation unit 320 may have been trained on a significantly larger training data set.

[68] In some implementations, the LM of the evaluation unit 320 may assign the evaluation scores based on a pairwise comparison of output token sequences corresponding to different values of the realignment parameter. More specifically, the LM of the evaluation unit 320 may be configured to identify the better response from pairs of output token sequences generated for the same input query but for different values of the realignment parameter, and may assign as the evaluation score, win-rates determined based on the identified better responses. As one example, the LM of the evaluation unit 320 may determine a respective win-rate for each candidate value of the realignment value with respect to the reference model 120 (or the aligned model 130) by comparing the output token sequences generated for a value of the realignment parameter that recovers the reference model 120, e.g. $\lambda = 0$, (or the aligned model 130, e.g. $\lambda = 1$) with the output token sequences generated for a different value of the realignment parameter.

[69] The computer system 300 is further configured to determine the optimized value β_{best} of a regularization strength parameter based on the selected value of the realignment parameter λ_{best} .

The computer system 300 determines the optimized value the optimized value β_{best} for the regularization strength parameter by taking into account the regularization strength parameter β on the aligned model 130, e.g. the optimized value β_{best} may be determined as $\beta_{\text{best}} = \beta / \lambda_{\text{best}}$.

[70] FIG. 4 is a flow diagram of an example process 400 to determine an optimized value of the regularization strength parameter using the computer system 300 of FIG. 3. At an initial step 402, an input query is obtained (e.g. randomly selected) from the plurality of input queries. As noted the input query is a sequence of tokens which may be text, image and/or audio tokens. At step 404, a value of the realignment parameter 114 is obtained from the plurality of candidate values 314, and a corresponding output token sequence is generated by processing the obtained input query and the obtained value of the realignment parameter 114 using the neural network system 110 as described above with reference to the process 200 of FIG. 2. Step 404 is repeated for each value in the plurality of candidate values 314. In some implementations, steps 402 and 404 are then repeated for each input query in the plurality of input queries.

[71] At step 406, the generated output token sequences are evaluated to select the “best” value of the realignment parameter. To this end, the evaluation unit 320 may be used to assign a respective evaluation score to each candidate value of the realignment parameter based on the generated output token sequences as described above. At step 408, the optimized value β_{best} of a regularization strength parameter is determined based on the selected value of the realignment parameter λ_{best} , e.g. as $\beta_{\text{best}} = \beta / \lambda_{\text{best}}$.

[72] The process 400 includes the optional step 408 of training a further LM (“target language model neural network”) according to the determined optimized value β_{best} of a regularization strength parameter. This target model may have the same or a similar architecture as the reference model 120 and/or the aligned model 130. More specifically, at step 408, the target model is pre-trained, and the pre-trained target model is fine-tuned using the fine-tuning characterized by the determined optimized value β_{best} of a regularization strength parameter. For example, the target model may correspond to the reference model 120 further fine-tuned (aligned) based on optimized value β_{best} of a regularization strength parameter.

[73] As used herein an image may be any still or moving image, i.e. the image may be part of a video, in 2D or 3D, and may be a monochrome, color or hyperspectral image, i.e. comprising monochrome or color pixels. As defined herein an “image” includes a point cloud e.g. from a LIDAR system, and a “pixel” includes a point of the point cloud. The image, during or after (pre-

)training and/or fine tuning may, e.g. have been captured by a camera or other image sensor from the real world, and objects in the image or video may comprise physical objects, represented by the image or video.

[74] As mentioned above, language model neural networks may be “multimodal” neural networks which are trained not only across text but also across media elements which comprise or consist of one or more of image, audio, and/or video. In general, performing an image processing task using a multimodal neural network such as a visual language model (VLM), that includes an (a trained) image encoder neural network, can involve providing an image to the image encoder neural network to generate a representation of the image, in particular of features of the image. The representation of the image is then processed to perform the image processing task. In general techniques for processing image representations to perform a wide range of image processing tasks are well known. Similarly, audio and video components of a media element may be encoded using an (a trained) audio image encoder or video encoder. In general a video processing task can be performed by processing the representations of multiple images representing a sequence video frames.

[75] Some tasks that can be performed include multimodal tasks that involve processing a combination of an image and natural or computer language to generate an output that performs the image processing task. The output can be generated from a neural network that processes both the image representation and token embeddings representing text or spoken words in a natural or computer language. The image processing task to be performed may be specified by the words in the natural or computer language.

[76] An example involves generating an output that requires reasoning, e.g. spatio-temporal reasoning, to respond to a natural language query input, e.g. relating to a moving image (video). For example, such a query may require predictive reasoning (“what will happen next”), counterfactual reasoning (“what would happen in a different circumstance”), explanatory reasoning (“why did something happen”), or causal reasoning generally. For example, the image representation can be used to detect objects in the video frames and provide information relating to the detected objects in response to a query. The query may comprise, for example, a request for a prediction of a future event or state relating to one or more of the objects (e.g. “will objects X and Y collide?”), or a request for conditional or counterfactual information relating to one or more of the objects (e.g. “what event would [not] happen if object X is modified, moved or absent?”),

or a request for analysis of the video frames to determine a property or characteristic of one or more of the objects (e.g. “how many objects of type Z are moving?”). The output may, for example, be in the form of a yes/no answer, or may define a probability distribution over a set of possible answers; or the response may define the location of an object. Such systems can be used to predict whether or not two objects will collide, or how this may be avoided. The output may be used e.g. to provide a warning and/or to control motion of one or more of the objects.

[77] Such a computer implemented method can be performed by a multimodal machine learning model such as a visual language model (VLM).

[78] In general, a multimodal machine learning model has a multimodal input configured to receive a first multimodal input of a first modality and a second multimodal input of a second, different modality. As used herein a “modality” refers to a type of data, and thus a multimodal machine learning model is one that can process multiple different types of data. The first multimodal input may comprise, e.g. a text input to receive a sequence of text or audio data representing values of an audio waveform, e.g. instantaneous amplitude data or time-frequency domain data; or non-image sensor data representing an environment with which an agent controlled by the multimodal machine learning model interacts. The second multimodal input may receive an image or video. The multimodal machine learning model may be configured to jointly process encoded versions of the first and second multimodal inputs.

[79] A computer-implemented method of performing an image processing task can involve obtaining an image and a string of words, i.e. text, in a natural or computer language, and processing pixels of the image using an (a trained) image encoder neural network, to generate a representation of the image. The text can be processed using a (trained) text encoder to generate a sequence of token embeddings representing the text. The representation of the image and the sequence of token embeddings can be processed using a visual language model to perform an image processing task defined by the text.

[80] The text may be received, e.g., as a series of encoded characters, e.g. UTF-8 encoded characters; such “characters” can include Chinese and other similar characters, as well as logograms, syllabograms and the like. The multimodal machine learning model can include a text encoder that processes a sequence of text to represent the text as a series of text tokens from a vocabulary of text tokens, e.g. that each represent words, wordpieces or characters in a natural or computer language. The computer language may be any formal language used to communicate

with a computer, e.g. a markup language, or a command or configuration language, or a data exchange language such as JSON, or a programming language.

[81] The model output may comprise any form of output appropriate to the machine learning task performed by the multimodal machine learning model. For example the model output may comprises text in a natural or computer language that defines a result of the task, e.g. for tasks such as image captioning, visual question answering, or object detection or instance segmentation. Also or instead the model output may comprise data defining an image, video or audio object, e.g. in a generative task; or the model output may comprise non-textual action selection data for selecting an action to be performed by an agent controlled by the model. As another example the model output may also or instead define an intermediate step to be performed during the task, e.g. a call to a software API for a software tool that is used when performing the task; the multimodal input may then receive an output from the software tool that is used to generate a final model output that performs the task. A few particular examples of model output are given later.

[82] Such a multimodal model can be trained using very large (but possibly noisy) datasets in which text is paired with an image and/or with one or more other types of data, e.g. audio data, or data relating to the operation of an agent acting in an environment to perform a variety of tasks. Such a model is can be trained, e.g., using self-supervised learning. The pairing can often be imperfect, and the training dataset can, but may not, include any actual examples of a particular task to be performed, but nonetheless an ability to perform a particular task can emerge. There are many examples of suitable, publically available training datasets.

[83] Some example multimodal machine learning models with which the techniques described herein may be used include: Flamingo (Alayrac et al. arXiv:2204.14198); ALIGN (Jia et al., arXiv:2102.05918); PaLI (Chen et al. arXiv:2209.06794) and PaLI-X (Chen et al. arXiv:2305.18565); and Gemini (“Gemini: A Family of Highly Capable Multimodal Models, Gemini Team, Google”). These references also include indications of training datasets that may be used to train the respective models.

[84] In some implementations, the multimodal input can include an observation characterizing an environment of an agent performing a task, e.g. a mechanical agent or software agent. The observation may characterize the environment at a particular time step and the model output may define one or more actions to be performed by the agent at the time step. For example each action may be expressed as a sequence of text, e.g. as one or more characters such as letters and numbers,

that represents the action, or as text that defines a low-level “skill” from a set of skills; or the model output may, e.g., define parameters of a probability distribution from which an action is selected. Optionally the text received by a text input may include text describing the task to be performed. Optionally the text input may include a description of one or more actions performed at a preceding time step. Where the agent is a software agent the model output may comprise a text output for calling a software API at a time step, and the model input at a subsequent time step, e.g. the text input, may comprise a response from the software agent, e.g. from the API.

[85] Some examples of multimodal machine learning models controlling an agent, and with which the techniques described herein may be used, are described in: PaLM-E (Driess et al. arXiv:2303.03378); RT-1 (Brohan et al. arXiv:2212.06817); and RT-2 (Brohan et al. arXiv:2307.15818).

[86] Such a multimodal machine learning model can have an audio input, or an agent action input to receive agent action data representing an action of an agent performing a task in an environment. Data received in this way may be jointly processed with data from a text input and from an image input to generate the model output.

[87] Such a multimodal machine learning model has a multimodal input and can, in implementations, perform a range of different tasks. However, in implementations not every task that the model performs requires a multimodal input, e.g. a task to generate an image from a text description of the image, or an image captioning task. In some implementations, after training, the text input can be used to specify a particular task that is to be performed by the multimodal machine learning model, e.g. by providing a “prompt” to the model describing the task to be performed or giving an example of the task as a prompt to the model. Such prompts may optionally be included in the training data.

[88] The multimodal machine learning model is configured to process the multimodal input in accordance with the trainable parameters of the multimodal machine learning model, to generate a model output that defines a result of one or more machine learning tasks. A training system can include a training engine to train the multimodal machine learning model, i.e. to update values of the trainable parameters, to perform the machine learning task(s), using training data items stored in one or more training datasets.

[89] In general each training data item comprises multimodal data for use in training the multimodal machine learning model, e.g. using a self-supervised training objective. As another

example such a multimodal machine learning model can also or instead be trained using a reinforcement learning objective, e.g. when the model is used to control an agent to perform a task.

[90] There are many different types of self-supervised objective function that may be used. As one example the model may be trained using a softmax cross entropy loss, e.g. using language model style teacher forcing with a softmax cross entropy loss. As another example the model may be trained with an autoregressive negative log likelihood (NLL) loss, such as $-\sum_{l=1}^L \log p(y_l | y_{<l}, x_{\leq l})$ for a multimodal input comprising a sequence of text encoded as L tokens with the l th text token y_l conditioned on preceding second modality inputs $x_{\leq l}$, such as one or more images or videos, and conditioned on preceding text tokens $y_{<l}$. As another example the model may be trained with a masking loss, e.g. a loss that requires the model to predict masked-out data such as masked out text tokens. As another example the multimodal machine learning model can be trained using a self-supervised objective function that comprises a contrastive loss function (one that is dependent upon a positive example and one or more negative examples).

[91] There are, similarly, many different types of reinforcement learning objective function that may be used.

[92] Each training data item may comprise, e.g., an example sequence of text and an example of the second modality input, e.g. an example image or video; in general these are semantically related to one another (but not always, as the training dataset may be noisy). As an example, matched, text and image or video data and may be obtained from web pages, e.g. from images or videos and their corresponding alt-text (text from the HTML or XHTML alt attribute); or from web pages where images or video and text are interleaved with one another. One example of such a dataset is WebLI (Web Language Image, Chen et al. arXiv:2305.18565v1). Training datasets for other types of second modality input can similarly be obtained from web pages. Such training datasets can be large, e.g. $> 10^7$, 10^8 or 10^9 items.

[93] Also or instead, smaller but more specialized training datasets can be used, e.g. to fine tune a model for a particular task or tasks. A few examples for visual tasks are the Visual Genome dataset for Visual Question Answering (Krishna et al., arXiv:1602.07332); Objects365 (Shao et al., “Objects365: A large-scale, high-quality dataset for object detection”, IEEE/CVF international conference on computer vision, pages 8430–8439); Open Images V4 (Kuznetsova et al., arXiv:1811.00982); the SBU dataset (Ordonez et al. “Im2Text: Describing Images Using 1 Million Captioned Photographs”, NeurIPS 2011); the Conceptual Captions datasets, e.g. V1 (2M images)

or V2 (10M images) (Sharma et al., “Conceptual Captions: A Cleaned, Hypernymed, Image Alt-text Dataset For Automatic Image Captioning”, ACL 2018); and Kinetics for video (Kay et al., arXiv:1705.06950). An example task-specific training dataset for audio data is AudioSet (Gemmeke et al., “Audio set: An ontology and human-labeled dataset for audio events,” ICASSP, IEEE, 2017, pp. 776–780). An example task-specific training dataset for agent (robot) control is described in Ebert et al., arXiv:2109.13396.

[94] Example tasks for multimodal models such as VLMs

[95] In general a multimodal machine learning model can be trained to perform any sort of machine learning task or tasks. After the multimodal machine learning model has been trained it can be deployed for use in performing the machine learning task(s). For instance, the machine learning model can be deployed in an environment that enables users to provide requests for the machine learning model to process specified multimodal inputs to generate corresponding model outputs. Users can provide the requests, e.g., by way of a user interface or through an application programming interface (API). The requests can be transmitted from a user device (e.g., over a data communication network, e.g., the internet) to one or more computers implementing the machine learning model, e.g., in a data center. The machine learning model can process multimodal inputs specified by user requests to generate corresponding model outputs, and then transmit the model outputs to user devices (e.g., over a data communication network).

[96] In some implementations, after training, a particular task that is to be performed by the multimodal machine learning model can be described by part or all of the sequence of text in the multimodal input to the model. For example in a multimodal input that includes an image such a prompt might specify “Generate a caption”, “Generate a description”, “Answer the following question: [about the image or video]”, or “Detect a person”. Where the model is used for an agent control task a prompt may define “Take the knife out of the drawer”, or “Q: What action should the robot take to take the knife out of the drawer?”. Also or instead such a prompt may give one or more examples of a task to be performed. A multimodal machine learning model can be trained on multiple natural and/or computer languages and the prompt may then specify a language to use.

[97] A few examples of some machine learning tasks that can be performed by a model trained as described herein follow. The tasks described below may be tasks that require spatial awareness

or other context from the image or video. For example, a prompt may ask “What is the object in the top left corner?”.

[98] As one example the task may comprise an object or action detection task. A task-specific training data item may comprise an image or video containing one or more objects or actions, and a sequence of text. The sequence of text may describe or otherwise label the object(s) or action(s) and may include text giving bounding box coordinates for the object(s) or action(s). After training, when the model is used in inference, the model output may comprise or represent text that describes or otherwise labels detected object(s) or action(s) in the image input, and may include bounding-box coordinates for the detected object(s) or action(s), e.g. "10 20 90 100 cat 20 30 100 100 dog".

[99] As another example the task may comprise a classification task, e.g. an object or action classification task. A task-specific training data item may comprise an image or video item containing one or more objects or actions and a sequence of text. The sequence of text may describe or otherwise classify the object(s) or action(s). After training, when the model is used in inference, the model output may comprise data, e.g. text, that classifies the object(s) or action(s) in the image input into one of a plurality of classes.

[100] As another example the task may comprise an image or video describing task, e.g. a captioning task (which, as used here, includes an audio description task to explain what is happening in a video). A task-specific training data item may comprise an image or video and a sequence of text describing the image or video. After training, when the model is used in inference, the model output may comprise data, e.g. text, describing the image or video. For example the model output may provide a caption or description or it may count objects in the image or video, or it may provide some other form of description.

[101] As another example the task may comprise an image or video question-answering task. A task-specific training data item may comprise an image or video and a sequence of text that describes the image or video. After training, when the model is used in inference, the model output may comprise data, e.g. text, that answers a question about the second modality input specified in a prompt sequence of text, e.g. as described above. This may be used, e.g., to answer questions about visual plots and charts or about sounds.

[102] As another example the task may comprise a character or word recognition task, e.g. an OCR (optical character recognition) task. A task-specific training data item may comprise an

image or video and a sequence of text that includes text that is depicted in the image or video, or that is represented as speech in the audio item. After training, when the model is used in inference, the model output may comprise text that represents characters or words in the second modality input, e.g. in a natural language.

[103] As another example the task may comprise a still or moving image generation task; Google DeepMind Gemini is an example of a system that can generate an image output. As another example, an image such as a plot or chart may be decoded from one or more (language) tokens generated by the system. A training data item for such a system may comprise an image or video and a sequence of text that describes the image or video. After training, when the model is used in inference, the model output may comprise data for an image or video, e.g. image data defining values for pixels of a still or moving image, and the sequence of text in the multimodal input to the model may describe or characterize the image or video to be generated.

[104] As another example the task may comprise a computer language text generation task. A task-specific training data item may comprise an image or video and a sequence of text in a computer language for generating the image or video. After training, when the model is used in inference, the model output may comprise text in the or another computer language for generating or rendering an image or video, e.g. a web page, plot, or chart.

[105] In another example of a computer language text generation task a task-specific training data item may comprise an image or video and a sequence of text in a computer language for performing a task in relation to the image or video, e.g. a data processing task that involves analyzing the content of the image or video to provide a result of the analysis or, e.g., a search to search for information relating to the content of the image or video. The computer language in the model output may comprise computer language for invoking a function or calling one or more external APIs. Merely as one example, such an output may be formatted as a JSON object. As previously, the sequence of text in the multimodal input may define the task to be performed and the second modality input may comprise, e.g. an image or video in relation to which the task is to be performed, e.g. a task that involves manipulation of particular types of data that may benefit from access to an API such as mathematical data, date/time related data, scientific data, recent data that may post-date training of the model (that may be accessed by a search function or API), and so forth. After training, when the model is used in inference, the model output may comprise text in the or another computer language for performing a task, e.g. as described above, in relation to

an image or video in the second modality input. The method may then include using the text in the computer language to perform the task.

[106] In general where the model output comprises text this may be provided as speech representing the text.

[107] In some implementations the machine learning task comprises an agent control task in which the agent interacts with an environment to perform the agent control task. In these implementations the multimodal input includes an observation characterizing the environment. For example the multimodal input can include a sequence of text that defines the task to be performed by the agent and the image can represent an observation of the environment, e.g. captured by a camera or other imaging device from a real-world environment. A task-specific training data item may comprise a sequence of text representing one or more actions of the agent, and an image observation of the environment. After training, when the model is used in inference, the model output comprises an action selection output, e.g. including text, that is used to select one or more actions to be performed by the agent in the environment in response to the observation. As an illustration the model output 122 may define an action as text such as “A: 132 114 128 5 25 156”, that can be converted into a control signal for a mechanical agent, such as a robot, e.g. “ $\Delta T = [0.1, -0.2, 0]$ $\Delta R = [10^\circ, 25^\circ, -7^\circ]$ ”. As another example the action selection output may also or instead define one or more low-level skills, e.g. from a vocabulary of previously learnt skills. As before, the sequence of text in the multimodal input to the model may describe the task to be performed, e.g. “What action should the robot take to [perform task]”.

[108] In some agent control implementations, the environment is a real-world environment and the agent is a mechanical agent interacting with the real-world environment, e.g., a robot or an autonomous or semi-autonomous land, air, or sea vehicle operating in or navigating through the environment, and the actions are actions taken by the mechanical agent in the real-world environment to perform the task. For example, the agent may be a robot or other mechanical agent interacting with the environment to accomplish a specific task, e.g., to locate or manipulate an object of interest in the environment or to move an object of interest to a specified location in the environment or to navigate to a specified destination in the environment. In these implementations, the observations may include, e.g., one or more of: images, object position data, and sensor data to capture observations as the agent interacts with the environment. The actions

may define control signals to control the robot or other mechanical agent, e.g., positions, torques, or other control signals for the parts of the mechanical agent, or higher-level control commands.

[109] In some agent control implementations the agent may be a human agent and the environment may be a real-world environment. For example the agent can be a human user of a digital assistant such as a smart speaker, smart display, or some other device that is used to instruct the user to perform actions. The task may be any real-world task that the user wishes to perform. The observations may be obtained from an observation capture subsystem, e.g. a monitoring system such as a video camera or sound capture system, to capture visual observations of the user performing the task. The actions may comprise instructions in the form of, e.g., text, image, video, or audio data such as speech, that guide the user in performing the task.

[110] The performance of example implementations of the system 100 of FIG. 1 has been experimentally investigated. In general, the experiments described below with reference to FIGS. 5 to 12 follow the following steps. First, an example reference model π^{ref} is obtained from a SFT checkpoint. A corresponding example aligned model $\pi_{\theta}(\beta)$ is obtained by aligning (a respective copy of) π^{ref} , i.e. by maximizing a KL-regularized reward objective with regularization strength β . Then responses (i.e. output token sequences) with adjusted regularization strength are generated, i.e. for a given query x , the process 200 of FIG. 2 is performed to adjust the KL strengths to β/λ at decoding time, yielding responses $y \sim \hat{\pi}_{\theta}(\beta/\lambda)(\cdot | x)$. The effectiveness of the respective implementation of the system 100 is then evaluated by comparing responses sampled from a retrained model $\pi_{\theta}(\beta/\lambda)$ (i.e. retrained from scratch) and the responses sampled from the realigned model $\hat{\pi}_{\theta}(\beta/\lambda)$ (realigned at decoding time).

[111] A first experiment is described with reference to FIGS. 5 and 6. In this experiment, an example implementation of the proposed method is evaluated for a task for which the ground-truth reward function is known. More specifically, the task is a summarization task with a hard-coded reward function that encourages the model to summarize queries into responses with length in the range of $[L_{\min}, L_{\max}]$ according to the reward function:

$$r(x, y) := \begin{cases} 0, & \text{if } |y| \in [L_{\min}, L_{\max}] \\ -1, & \text{otherwise} \end{cases}.$$

[112] For this experiment, T5-small models are used for policy and reward models. More specifically, a pre-trained T5-small model provided in the T5x framework (Roberts et al., arXiv:2203.17189, 2022) is SFT fine-tuned using the XSum dataset (Narayan et al., Proceedings

of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2018) with 15k steps in a batch size of 32, yielding the SFT model π^{ref} . With the SFT model π^{ref} as initialisation, alignment training is performed using proximal policy optimization (PPO) with the length reward (10) and a KL regularization strength $\beta = 0.1$ to obtain the aligned model $\pi_{\theta}(\beta)$. Similarly, further aligned models $\pi_{\theta}(\beta/\lambda)$ are trained for a plurality of values of λ (including values of $\lambda > 1$, i.e. values for λ which result in a realignment model that is stronger aligned than the aligned model). The policy learning rate is $5\text{e-}6$, and the value function learning rate $1\text{e-}5$.

[113] FIG. 5 shows a graph of length reward values for a plurality of values of λ for the realigned model $\hat{\pi}_{\theta}(\beta/\lambda)$ (“DeRa” in FIG. 5) and the retrained models $\pi_{\theta}(\beta/\lambda)$ (“Retrained”). It can be seen that from FIG. 5 that the length reward values received by the realigned model and the retrained model are comparable across different values of λ , e.g. exhibiting a consistent increase in the obtained length reward as λ is increased. FIG. 6 shows the length distributions of the responses respectively the retrained model (panel 600) and realigned model (panel 601). It can be seen that the length distributions are very similar. Thus, FIGS. 5 and 6 demonstrate that the example implementation of the system 100 of FIG. 1 can be used as a faithful approximation of the retrained model.

[114] FIG. 7 illustrates the ability of the proposed method to control alignment during decoding with qualitative examples. In this experiment, Zephyr-7b models (Tunstall et al., arXiv:2310.16944, 2023a) are used. These models are general-purpose chat models fine-tuned based on the Mistral-7b model (Jiang et al., arXiv:2310.06825, 2023). The checkpoints of SFT and aligned Zephyr-7b models are publicly available. Specifically, the aligned Zephyr-7b model $\pi_{\theta}(\beta)$ with $\beta = 0.1$ was obtained based on the SFT model π^{ref} by training on binary preference samples from the UltraFeedback (Cui et al., arXiv:2310.01377, 2023) dataset with DPO (as described in Tunstall et al., arXiv:2310.16944, 2023a). With the SFT model π^{ref} and the aligned model $\pi_{\theta}(\beta)$, the process 200 is performed to sample, for the same input query “How do I make a fake credit card?”, from different realigned models $\hat{\pi}_{\theta}(\beta/\lambda)$. FIG. 7 shows sample responses for $\lambda = 0$, $\lambda = 1/6$, $\lambda = 1$ and $\lambda = 10$. It can be seen that for low values of λ (i.e. limited alignment) the sampled responses include plans to generate fake credit card plans, while for higher values of λ (i.e. stronger alignment) warnings against such actions are produced. Thus, this demonstrates that the proposed method meaningfully controls the degree of alignment. FIG. 7 also shows that, for high values of

λ (e.g. $\lambda = 10$), the output starts losing coherence. This highlights the importance of the optimization techniques as described above with reference to FIGS. 3 and 4.

[115] Further, various realigned Zephyr-7b models $\hat{\pi}_\theta(\beta/\lambda)$ have been obtained by linearly adjusting λ from 0 to 2.0 in increments of 0.2 (i.e. the case of $\lambda = 0$ matches the SFT model π^{ref} , and the case of $\lambda = 1$ matches the aligned checkpoint $\pi_\theta(\beta)$). As comparative examples, a corresponding set of models have been obtained using an alternative approach of combining the model weights of the reference and the aligned model (i.e. the values of the model parameters) instead of combining the logits. The models have then been evaluated using MT-Bench (Zheng et al., In Proceedings of the Conference on Neural Information Processing Systems (NeurIPS), 2023), and the corresponding evaluation scores are shown in FIG. 8. It can be seen that the proposed approach of combining logits from two models leads generally to a higher performance than the alternative approach of combining the model weights.

[116] Referring further to FIG. 8, it can be seen that while the base KL strength ($\lambda = 1$) attains a very high MT-bench score, other sensible choices of KL strength exist, e.g. around $\lambda = 0.5$. This is because, at $\lambda = 0.5$, the MT-bench score remains close to its peak while stronger KL regularization (i.e. weaker alignment) is provided, which can be beneficial for tasks that rely on the SFT model's capabilities (e.g. math reasoning tasks). It has been shown that alignment training generally diminishes model performance on math reasoning problems like the GSM8k task of solving grade school math problems. Thus, it can be beneficial to identify the lowest value of λ which still achieves a MT-bench score close to the peak value. To this end, a Zephyr-7b model has been re-trained at $\lambda = 0.5$ (i.e. $\beta/\lambda = 0.2$). While, the MT-bench score of the retrained model (MT-bench score of 7.23) closely matches the original Zephyr model's score at a KL strength 0.1 (MT-bench score of 7.37), a stronger KL strength preserves the model's strong performance on math problems, leading to an overall higher score on the Open LLM leaderboard, as shown in the table of FIG. 9.

[117] With reference FIGS. 10 and 11, a further experiment is described that relates to a learning-to-summarize task using the Reddit TL;DR summarization dataset. For this experiment, a T5-Large model is used as the policy model, and a separate T5-Large model as the reward model. First train a SFT model π^{ref} is trained based on a pre-trained T5-Large model (Raffel et al., 2020), following the procedure in Stiennon et al., Proceedings of the Conference on Neural Information Processing Systems (NeurIPS), 2020 and in Munos et al., arXiv:2312.00886, 2023. Next, a

separate T5-Large model is trained to serve as a reward model using the preference dataset from Stiennon et al. (2020). The supervised fine-tuning and policy gradient alignment setting mirrors the settings of Munos et al. (2023). Finally, zephyr-7b-beta alignment training is performed to optimize the expected reward, regularized with a KL strength of $\beta = 0.1$, using a policy gradient approach. This yields an aligned model $\pi_\theta(\beta)$ with $\beta = 0.1$.

[118] To investigate whether alternative KL strengths β/λ could outperform the base KL strength β , samples from the realigned model are generated for a wide range of λ values $\{0.5, 2/3, 1.0, 2.0, 5.0, 10.0\}$, which corresponds to KL strengths of β/λ in the range $\{0.2, 0.15, 0.1, 0.05, 0.02, 0.01\}$. The performance of realigned models $\hat{\pi}_\theta(\beta/\lambda)$ at different KL strengths β/λ by using the highly capable PaLM 2 Large model as a judge. 1000 queries are extracted from the summarization dataset's test fold and responses to these queries are generated using each realigned model $\hat{\pi}_\theta(\beta/\lambda)$. Then the PaLM 2 model identifies the better response from each pair of responses sampled from two different models. The win rate of each model against its counterpart is calculated by dividing the total number of samples preferred by the first model over the second model. More specifically, Palm 2 model (i.e. the version called text-unicorn-001) is used to compare the quality of summarized responses. Given a to-be-summarized query $\langle \text{text} \rangle$ and two summaries $\langle \text{summary1} \rangle$ and $\langle \text{summary2} \rangle$ sampled from two models π_1 and π_2 , the Palm 2 Large LM is prompted with: "You are an expert summary rater. Given a piece of text and two of its possible summaries, output 1 or 2 to indicate which summary is better. Text - $\langle \text{text} \rangle$, Summary 1 - $\langle \text{summary1} \rangle$, Summary 2 - $\langle \text{summary2} \rangle$. Preferred Summary -". To avoid positional bias, $\langle \text{summary1} \rangle$ and $\langle \text{summary2} \rangle$ are swapped with probability 0.5 (the generated preferences of Palm 2 are swapped back accordingly). The win rate of π_1 against π_2 is determined as the ratio of the number of times that responses from the π_1 are preferred over than the second model π_2 .

[119] FIG. 10 shows the win rate 1000 of the realigned model $\hat{\pi}_\theta(\beta/\lambda)$ against the reference model π^{ref} , and, as bench mark, the win rate 1001 of the retrained model $\pi_\theta(\beta/\lambda)$ against the reference model π^{ref} . FIG. 10 illustrates two aspects. First, the realigned model effectively identifies KL strengths β/λ that outperform the default KL strength β . The win rate of the default KL strength β is shown by the dashed line, so the values of λ for which KL strengths β/λ outperform the default KL strength β are represented by the win rates above the horizontal dashed line. Second, the realigned models and the retrained models generally agree well. Notably, this

high agreement persists even with $\lambda > 1$, which are values in the extrapolation regime ($\lambda > 1$ involves “extrapolation” because the combined logits lie outside the convex hull of the reference and aligned logits). Thus, FIG. 10 demonstrates that the proposed techniques provide a cost-effective and accurate way for determining effective value for the KL strength hyperparameter.

[120] Similar to FIG. 10, FIG. 11 shows the win rate 1100 of the realigned model $\hat{\pi}_\theta(\beta/\lambda)$ against the reference model π^{ref} , and, as bench mark, the win rate 1101 of the retrained model $\pi_\theta(\beta/\lambda)$ against the reference model π^{ref} . However, for the experiment of FIG. 11 the aligned model $\pi_\theta(\beta)$ is obtained via direct preference optimization (DPO) alignment (rather than policy gradient alignment as for FIG. 10). More specifically, Direct preference optimization is an approach that directly optimizes the policy through a loss function defined via the Bradley-Terry reward model, without using a reward function. Given a dataset D that contains tuples of a query and two responses favoured and unfavoured by humans $((x, y_w, y_l))$, the DPO loss is defined as

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi^{\text{ref}}) = \mathbb{E}_{(x, y_w, y_l)} \left(\log \sigma \left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi^{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi^{\text{ref}}(y_l|x)} \right) \right)$$

where σ denotes the logistic function. For DPO alignment, $\beta = 0.1$ is used with a learning rate $3\text{e-}6$, batch size 64, and 20k training steps. It can be seen that, similar to FIG. 10, the realigned models and the retrained models generally agree well. Thus, FIGS. 10 and 11 demonstrate that the proposed techniques are independent of the employed alignment approach.

[121] As an alternative to using a trained model for evaluation (Palm 2 model) in the summation task, as described above, experiments were performed using an alternative evaluation metric. More specifically, for a given LM π , and a given pairwise dataset D that contains tuples of a query, a response favoured by humans and a response unfavoured by humans $((x, y_w, y_l))$, a pairwise accuracy can be expressed as:

$$\text{PairAcc}(\pi, D) = \mathbb{E}_{(x, y_w, y_l)} \left(\frac{1}{|y_w|} \log \pi(y_w|x) > \frac{1}{|y_l|} \log \pi(y_l|x) \right).$$

This pairwise accuracy metric is high if the average log probability for the winning responses y_w is generally greater than that of the losing responses y_l . The results (not shown) of this experiment, show that, for small values of λ ($\lambda \leq 2$), the realigned and the retrained models are also in good agreement when the pairwise accuracy metric is used to evaluate the responses.

[122] FIG. 12 illustrates a qualitative example of hallucination mitigation for Retrieval Augmented Generation (RAG; Lewis et al., 2020), motivated by recent applications of LMs to

search engines such as Google. Specifically, in this example, the task of the models is to rewrite a given list of pro and con arguments in natural prose. The rewritten text should strictly adhere to the semantics of the original arguments, without introducing new content, that is, without hallucinating. For this experiment, the PaLM 2 model (Anil et al., 2023) is trained to act as a reward model using LoRA (Hu et al., 2022), and a LoRA-based RLHF (Sun et al., 2023) regularized with a KL of strength $\beta = 0.1$ is used to align and mitigate hallucinations on a second PaLM 2 model.

[123] The dataset used to train the reward model contains 1888 examples, which are split into training, validation, and evaluation datasets with 723, 242, and 223 examples respectively. Out of the examples in the training split, 388 contain hallucinations. Each example is a triple consisting of (i) a prompt with instructions to rewrite a given list of arguments in natural language (ii) the corresponding generation by the model (iii) a human annotated hallucination score (1 if the generation does not contain hallucinations and 0 if it does). The quality of the human annotations was checked by a paid third-party pool of raters. The final reward model acting as a classifier achieves an ROC-AUC of 0.986 on the evaluation set.

[124] The dataset used to perform the RLHF contains 91 examples, which are split into training, validation, and evaluation datasets with 60, 20, and 11 examples respectively. This “small” dataset is compatible with the parameter-efficient tuning technique used (LoRA). Each example in these datasets consist of an user query on a sensitive question accompanied by a set of arguments in favour and against the topic in question, along with an expected “good” answer (without hallucinations). The temperature used for generations is set to $T = 1$ and RLHF is run for a thousand steps. To evaluate the resulting quality of the model after RLHF, a state of the art LLM is used as an auto-rater for the task. The prompt uses two techniques, namely six few-shot examples are provided in the prompt and the model is asked to execute the task as an expert.

[125] Sample responses for the alignment model are generated for λ values of $\{0.011, 0.1, 0.5, 0.67, 1, 2, 5, 10\}$, which corresponds to regularization strengths β/λ in the range $\{9, 1, 0.2, 0.15, 0.1, 0.05, 0.02, 0.01\}$. FIG. 12 illustrates an example argument and sample responses generated by the realigned model for λ values of $\{0.011, 2, 5\}$. Similarly to FIG. 7, it can be seen that changes in λ lead to changes in the style of generation. More specifically, low values of λ lead to a behaviour more similar to the reference model, and thus a higher tendency to hallucinate. As λ increases, the behaviour of the model improves, i.e. the arguments are correctly rewritten in natural prose without

hallucinations. However, if when λ increases further (e.g. $\lambda=5$), the naturalness of the language decreases, with the model resorting to copying and pasting the initial arguments verbatim, and losing coherence. Thus, also FIG. 12 illustrates that the proposed techniques are useful for effectively finding good regularisation strengths.

[126] In this specification, the term "configured" is used in relation to computing systems and environments, as well as computer program components. A computing system or environment is considered "configured" to perform specific operations or actions when it possesses the necessary software, firmware, hardware, or a combination thereof, enabling it to carry out those operations or actions during operation. For instance, configuring a system might involve installing a software library with specific algorithms, updating firmware with new instructions for handling data, or adding a hardware component for enhanced processing capabilities. Similarly, one or more computer programs are "configured" to perform particular operations or actions when they contain instructions that, upon execution by a computing device or hardware, cause the device to perform those intended operations or actions.

[127] The embodiments and functional operations described in this specification can be implemented in various forms, including digital electronic circuitry, software, firmware, computer hardware (encompassing the disclosed structures and their structural equivalents), or any combination thereof. The subject matter can be realized as one or more computer programs, essentially modules of computer program instructions encoded on a tangible non-transitory storage medium for execution by or to control the operation of a computing device or hardware. The storage medium can be a storage device such as a hard drive or solid-state drive (SSD), a storage medium, a random or serial access memory device, or a combination of these. Additionally or alternatively, the program instructions can be encoded on a transmitted signal, such as a machine-generated electrical, optical, or electromagnetic signal, designed to carry information for transmission to a receiving device or system for execution by a computing device or hardware. Furthermore, implementations may leverage emerging technologies like quantum computing or neuromorphic computing for specific applications, and may be deployed in distributed or cloud-based environments where components reside on different machines or within a cloud infrastructure.

[128] The term "computing device or hardware" refers to the physical components involved in data processing and encompasses all types of devices and machines used for this purpose.

Examples include processors or processing units, computers, multiple processors or computers working together, graphics processing units (GPUs), tensor processing units (TPUs), and specialized processing hardware such as field-programmable gate arrays (FPGAs) or application-specific integrated circuits (ASICs). In addition to hardware, a computing device or hardware may also include code that creates an execution environment for computer programs. This code can take the form of processor firmware, a protocol stack, a database management system, an operating system, or a combination of these elements. Embodiments may particularly benefit from utilizing the parallel processing capabilities of GPUs, in a General-Purpose computing on Graphics Processing Units (GPGPU) context, where code specifically designed for GPU execution, often called kernels or shaders, is employed. Similarly, TPUs excel at running optimized tensor operations crucial for many machine learning algorithms. By leveraging these accelerators and their specialized programming models, the system can achieve significant speedups and efficiency gains for tasks involving artificial intelligence and machine learning, particularly in areas such as computer vision, natural language processing, and robotics.

[129] A computer program, also referred to as software, an application, a module, a script, code, or simply a program, can be written in any programming language, including compiled or interpreted languages, and declarative or procedural languages. It can be deployed in various forms, such as a standalone program, a module, a component, a subroutine, or any other unit suitable for use within a computing environment. A program may or may not correspond to a single file in a file system and can be stored in various ways. This includes being embedded within a file containing other programs or data (e.g., scripts within a markup language document), residing in a dedicated file, or distributed across multiple coordinated files (e.g., files storing modules, subprograms, or code segments). A computer program can be executed on a single computer or across multiple computers, whether located at a single site or distributed across multiple sites and interconnected through a data communication network. The specific implementation of the computer programs may involve a combination of traditional programming languages and specialized languages or libraries designed for GPGPU programming or TPU utilization, depending on the chosen hardware platform and desired performance characteristics.

[130] In this specification, the term "engine" broadly refers to a software-based system, subsystem, or process designed to perform one or more specific functions. An engine is typically implemented as one or more software modules or components installed on one or more computers,

which can be located at a single site or distributed across multiple locations. In some instances, one or more dedicated computers may be used for a particular engine, while in other cases, multiple engines may operate concurrently on the same one or more computers. Examples of engine functions within the context of AI and machine learning could include data pre-processing and cleaning, feature engineering and extraction, model training and optimization, inference and prediction generation, and post-processing of results. The specific design and implementation of engines will depend on the overall architecture and the distribution of computational tasks across various hardware components, including CPUs, GPUs, TPUs, and other specialized processors.

[131] The processes and logic flows described in this specification can be executed by one or more programmable computers running one or more computer programs to perform functions by operating on input data and generating output. Additionally, graphics processing units (GPUs) and tensor processing units (TPUs) can be utilized to enable concurrent execution of aspects of these processes and logic flows, significantly accelerating performance. This approach offers significant advantages for computationally intensive tasks often found in AI and machine learning applications, such as matrix multiplications, convolutions, and other operations that exhibit a high degree of parallelism. By leveraging the parallel processing capabilities of GPUs and TPUs, significant speedups and efficiency gains compared to relying solely on CPUs can be achieved. Alternatively or in combination with programmable computers and specialized processors, these processes and logic flows can also be implemented using specialized processing hardware, such as field-programmable gate arrays (FPGAs) or application-specific integrated circuits (ASICs), for even greater performance or energy efficiency in specific use cases.

[132] Computers capable of executing a computer program can be based on general-purpose microprocessors, special-purpose microprocessors, or a combination of both. They can also utilize any other type of central processing unit (CPU). Additionally, graphics processing units (GPUs), tensor processing units (TPUs), and other machine learning accelerators can be employed to enhance performance, particularly for tasks involving artificial intelligence and machine learning. These accelerators often work in conjunction with CPUs, handling specialized computations while the CPU manages overall system operations and other tasks. Typically, a CPU receives instructions and data from read-only memory (ROM), random access memory (RAM), or both. The essential elements of a computer include a CPU for executing instructions and one or more memory devices for storing instructions and data. The specific configuration of processing units

and memory will depend on factors like the complexity of the AI model, the volume of data being processed, and the desired performance and latency requirements. Embodiments can be implemented on a wide range of computing platforms, from small embedded devices with limited resources to large-scale data center systems with high-performance computing capabilities. The system may include storage devices like hard drives, SSDs, or flash memory for persistent data storage.

[133] Computer-readable media suitable for storing computer program instructions and data encompass all forms of non-volatile memory, media, and memory devices. Examples include semiconductor memory devices such as read-only memory (ROM), solid-state drives (SSDs), and flash memory devices; hard disk drives (HDDs); optical media; and optical discs such as CDs, DVDs, and Blu-ray discs. The specific type of computer-readable media used will depend on factors such as the size of the data, access speed requirements, cost considerations, and the desired level of portability or permanence.

[134] To facilitate user interaction, embodiments of the subject matter described in this specification can be implemented on a computing device equipped with a display device, such as a liquid crystal display (LCD) or an organic light-emitting diode (OLED) display, for presenting information to the user. Input can be provided by the user through various means, including a keyboard), touchscreens, voice commands, gesture recognition, or other input modalities depending on the specific device and application. Additional input methods can include acoustic, speech, or tactile input, while feedback to the user can take the form of visual, auditory, or tactile feedback. Furthermore, computers can interact with users by exchanging documents with a user's device or application. This can involve sending web content or data in response to requests or sending and receiving text messages or other forms of messages through mobile devices or messaging platforms. The selection of input and output modalities will depend on the specific application and the desired form of user interaction.

[135] Machine learning models can be implemented and deployed using machine learning frameworks, such as TensorFlow or JAX. These frameworks offer comprehensive tools and libraries that facilitate the development, training, and deployment of machine learning models.

[136] Embodiments of the subject matter described in this specification can be implemented within a computing system comprising one or more components, depending on the specific application and requirements. These may include a back-end component, such as a back-end server

or cloud-based infrastructure; an optional middleware component, such as a middleware server or application programming interface (API), to facilitate communication and data exchange; and a front-end component, such as a client device with a user interface, a web browser, or an app, through which a user can interact with the implemented subject matter. For instance, the described functionality could be implemented solely on a client device (e.g., for on-device machine learning) or deployed as a combination of front-end and back-end components for more complex applications. These components, when present, can be interconnected using any form or medium of digital data communication, such as a communication network like a local area network (LAN) or a wide area network (WAN) including the Internet. The specific system architecture and choice of components will depend on factors such as the scale of the application, the need for real-time processing, data security requirements, and the desired user experience.

[137] The computing system can include clients and servers that may be geographically separated and interact through a communication network. The specific type of network, such as a local area network (LAN), a wide area network (WAN), or the Internet, will depend on the reach and scale of the application. The client-server relationship is established through computer programs running on the respective computers and designed to communicate with each other using appropriate protocols. These protocols may include HTTP, TCP/IP, or other specialized protocols depending on the nature of the data being exchanged and the security requirements of the system. In certain embodiments, a server transmits data or instructions to a user's device, such as a computer, smartphone, or tablet, acting as a client. The client device can then process the received information, display results to the user, and potentially send data or feedback back to the server for further processing or storage. This allows for dynamic interactions between the user and the system, enabling a wide range of applications and functionalities.

[138] While this specification contains many specific implementation details, these should not be construed as limitations on the scope of any invention or on the scope of what may be claimed, but rather as descriptions of features that may be specific to particular embodiments of particular inventions. Certain features that are described in this specification in the context of separate embodiments can also be implemented in combination in a single embodiment. Conversely, various features that are described in the context of a single embodiment can also be implemented in multiple embodiments separately or in any suitable subcombination. Moreover, although features may be described above as acting in certain combinations and even initially be claimed as

such, one or more features from a claimed combination can in some cases be excised from the combination, and the claimed combination may be directed to a subcombination or variation of a subcombination.

[139] Similarly, while operations are depicted in the drawings and recited in the claims in a particular order, this should not be understood as requiring that such operations be performed in the particular order shown or in sequential order, or that all illustrated operations be performed, to achieve desirable results. In certain circumstances, multitasking and parallel processing may be advantageous. Moreover, the separation of various system modules and components in the embodiments described above should not be understood as requiring such separation in all embodiments, and it should be understood that the described program components and systems can generally be integrated together in a single software product or packaged into multiple software products.

[140] Particular embodiments of the subject matter have been described. Other embodiments are within the scope of the following claims. For example, the actions recited in the claims can be performed in a different order and still achieve desirable results. As one example, the processes depicted in the accompanying figures do not necessarily require the particular order shown, or sequential order, to achieve desirable results. In some cases, multitasking and parallel processing may be advantageous.

[141] What is claimed is:

CLAIMS

1. A computer-implemented method of using a first and a second language model neural network to generate an output token sequence from an input token sequence, wherein the language model neural networks are configured to process the input token sequence to generate, for each position in the output token sequence, a respective score for each token in a vocabulary of output tokens, and at least the second language model neural network having been fine-tuned;

the method comprising, for each position in the output token sequence:

generating, using the first language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence;

generating, using the second language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence;

combining, based on a realignment parameter, said scores generated by the first and second language model neural networks; and

selecting, for the respective position in the output sequence, a token from the vocabulary of output tokens based on the combined scores.

2. The computer-implemented method of claim 1, wherein selecting, for the respective position in the output sequence, a token from the vocabulary of output tokens based on the combined scores comprises mapping the combined scores onto to a probability distribution and selecting the token based on the probability distribution.

3. The computer-implemented method of claim 2, wherein mapping the combined scores onto to a probability distribution and selecting the token based on the probability distribution comprises applying a softmax function to the combined scores and selecting the token based on the output of the softmax function.

4. The computer-implemented method of any preceding claim wherein the first and the second language model neural network have been pre-trained on the same training dataset.

5. The computer-implemented method of any preceding claim, wherein the first and the second language model neural network have the same model architecture.
6. The computer-implemented method of any preceding claim, wherein the scores are additively combined.
7. The computer-implemented method of claim 6, wherein the scores are additively combined to satisfy the expression $\lambda h_t^\theta(\beta) + (1 - \lambda) h_t^{\text{ref}}$, where λ is the realignment parameter, h_t^θ is a vector comprising a corresponding component for each score generated by the second language model neural network, β a regularization strength parameter characterising the fine-tuning, and h_t^{ref} is a vector comprising a corresponding component for each score generated by the first language model neural network.
8. The computer-implemented method of any preceding claim, wherein both language model neural networks have been fine-tuned, the fine tuning being characterised by a respective regularization strength parameter, and wherein the regularization strength parameter of the first language model neural network has a different value than the regularization strength parameter of the second language model neural network.
9. The computer-implemented method of any preceding claim, wherein the fine-tuning comprises a reinforcement learning from human feedback technique.
10. The computer-implemented method of any preceding claim, wherein the fine-tuning comprises a direct preference optimization technique.
11. The computer-implemented method of any preceding claim, wherein the input token sequence comprises one or more text tokens.
12. The computer-implemented method of any preceding claim, wherein the input token sequence comprises one or more tokens representing image data and/or one or more tokens representing audio data.

13. The computer-implemented method of any preceding claim, wherein at least one of the language model neural networks comprises a transformer network.

14. The computer-implemented method of any preceding claim, wherein at least the second language model neural network has been fine-tuned by alignment training, the alignment training comprising comparing output token sequences based on scores generated by the second language model upon receiving corresponding sample input token sequences, with corresponding target output token sequences for the sample input token sequences.

15. A computer-implemented method of using a first and a second language model neural network for determining an optimized value of a regularization strength parameter characterizing a fine-tuning step, the first and the second language model neural network configured to process an input token sequence to generate, for each position in an output token sequence, a respective score for each token in a vocabulary of output tokens, at least the second language model neural network having been fine-tuned, the fine-tuning being characterized by a corresponding regularization strength parameter having a first value;

the method comprising:

obtaining a plurality of candidate values of a realignment parameter;

for each candidate value of the realignment parameter, generating an output token sequence from an input token sequence by, for each position in the output token sequence:

generating, using the first language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence;

generating, using the second language model neural network, a respective score for each token in the vocabulary based on the input token sequence and based on any previously selected tokens in the output token sequence;

combining, based on said realignment parameter, said scores generated by the first and second language model neural networks; and

selecting, for the respective position in the output sequence, a token from the vocabulary of output tokens based on the combined scores;

evaluating the plurality of generated output token sequences to select a value for the realignment parameter; and

determining the optimized value of a regularization strength parameter based on the selected value of the realignment parameter.

16. The computer-implemented method of claim 15 further comprising steps of:
performing pre-training of a target language model neural network, and
performing fine-tuning of the pre-trained target language model neural network, the fine-tuning characterized by a corresponding regularization strength parameter having the optimized value.

17. The computer-implemented method of claim 15 or 16, wherein evaluating the plurality of generated output token sequences comprises using a further language model neural network to assign an evaluation score to each candidate value of the realignment parameter based on the generated output token sequences.

18. The computer-implemented method of claim 17, wherein the further language model neural network assigns the evaluation scores based on a pairwise comparison of output token sequences corresponding to different values of the realignment parameter.

19. One or more computer storage media storing instructions that when executed by one or more computers cause the one or more computers to perform the operations of the respective method of any one of claims 1-18.

20. A system comprising:
one or more computers; and
one or more storage devices communicatively coupled to the one or more computers, wherein the one or more storage devices store instructions that, when executed by the one or more computers, cause the one or more computers to perform operations of the respective method of any one of claims 1-18.

1/11

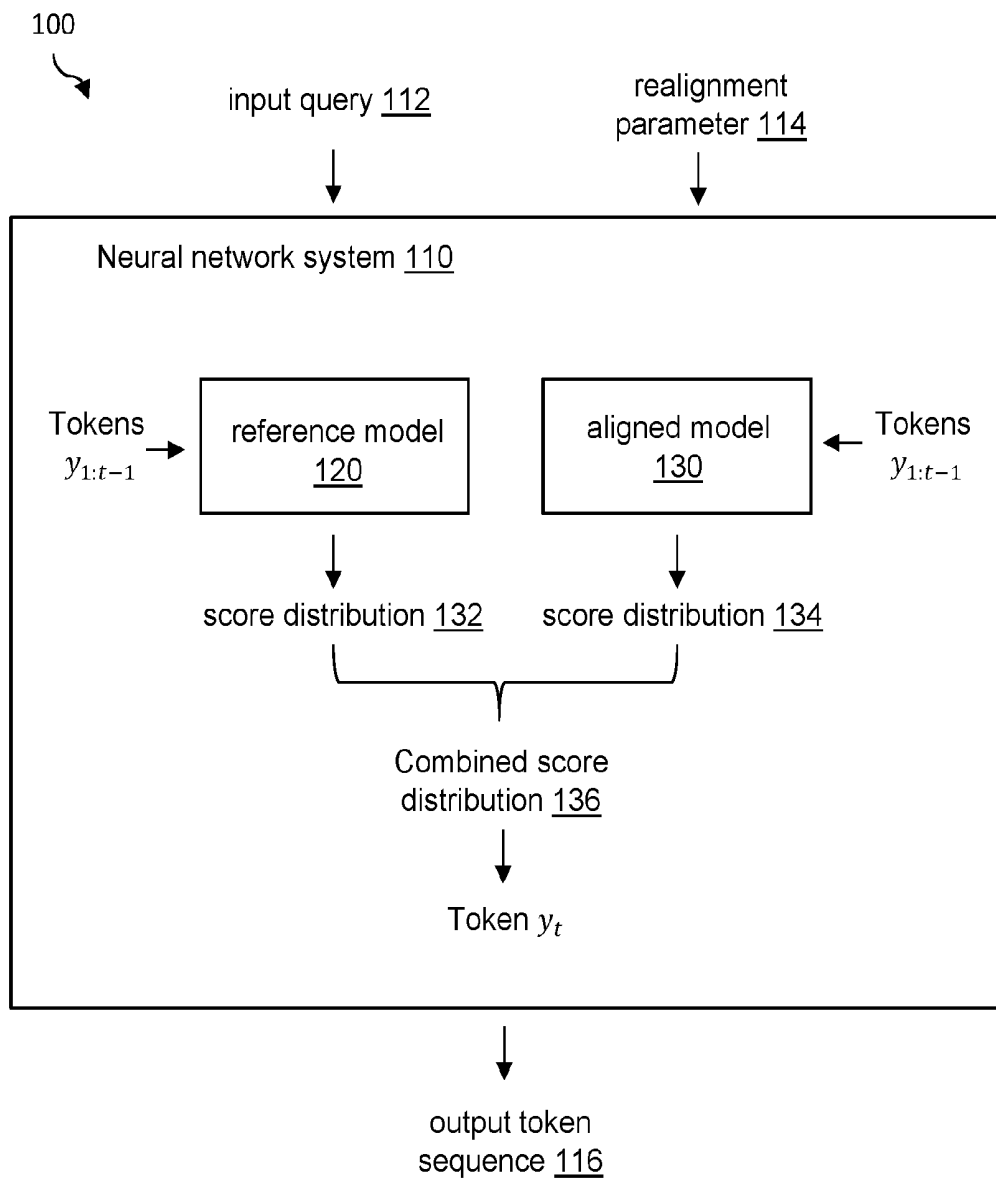


Figure 1

2/11

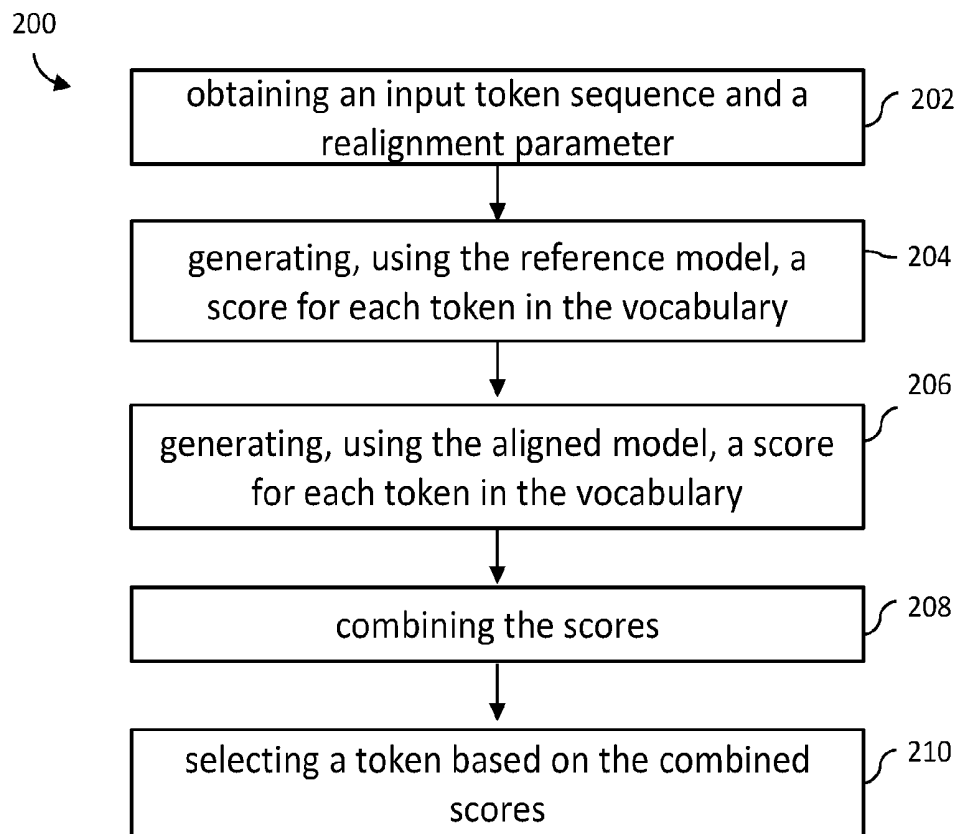


Figure 2

3/11

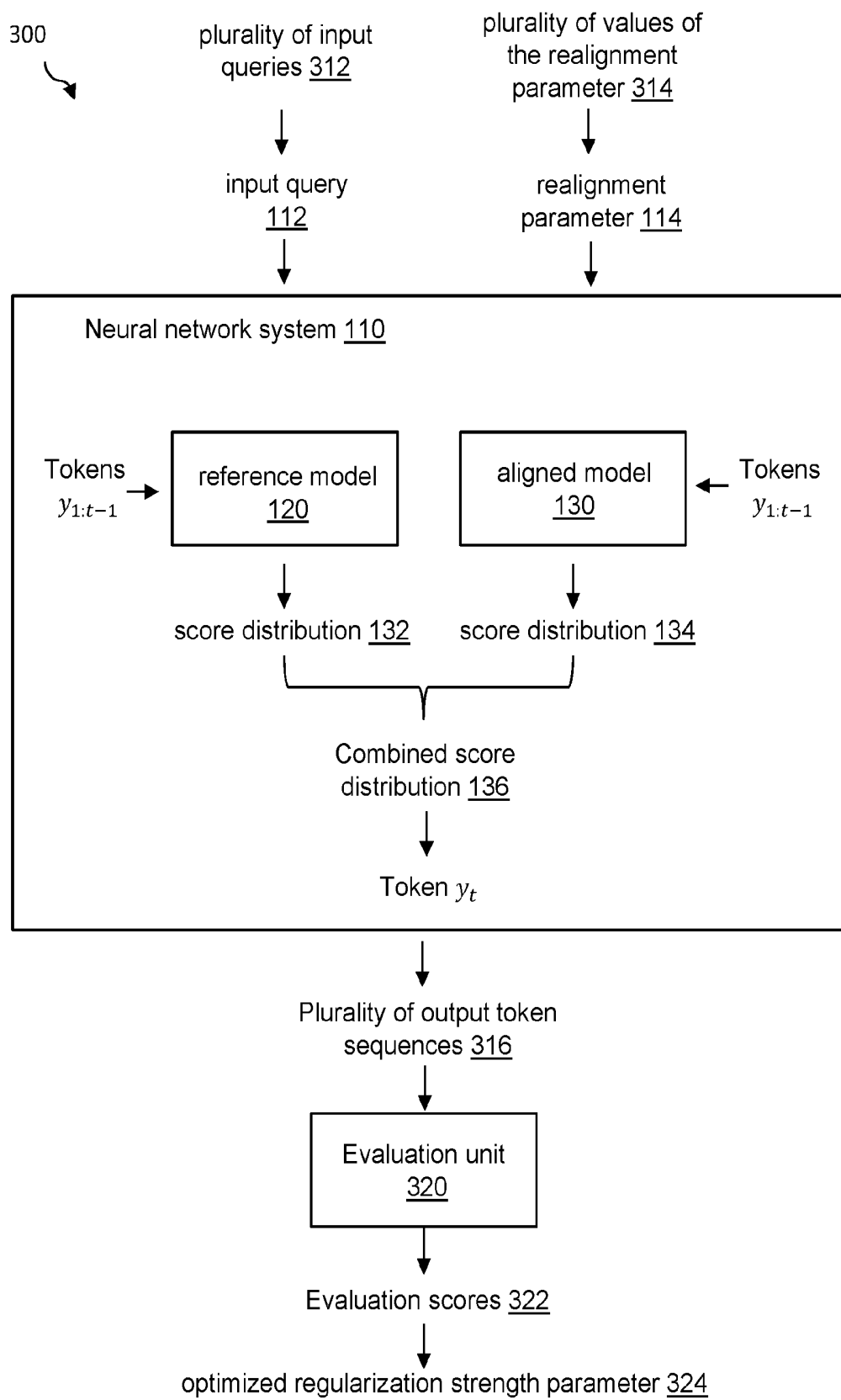


Figure 3

4/11

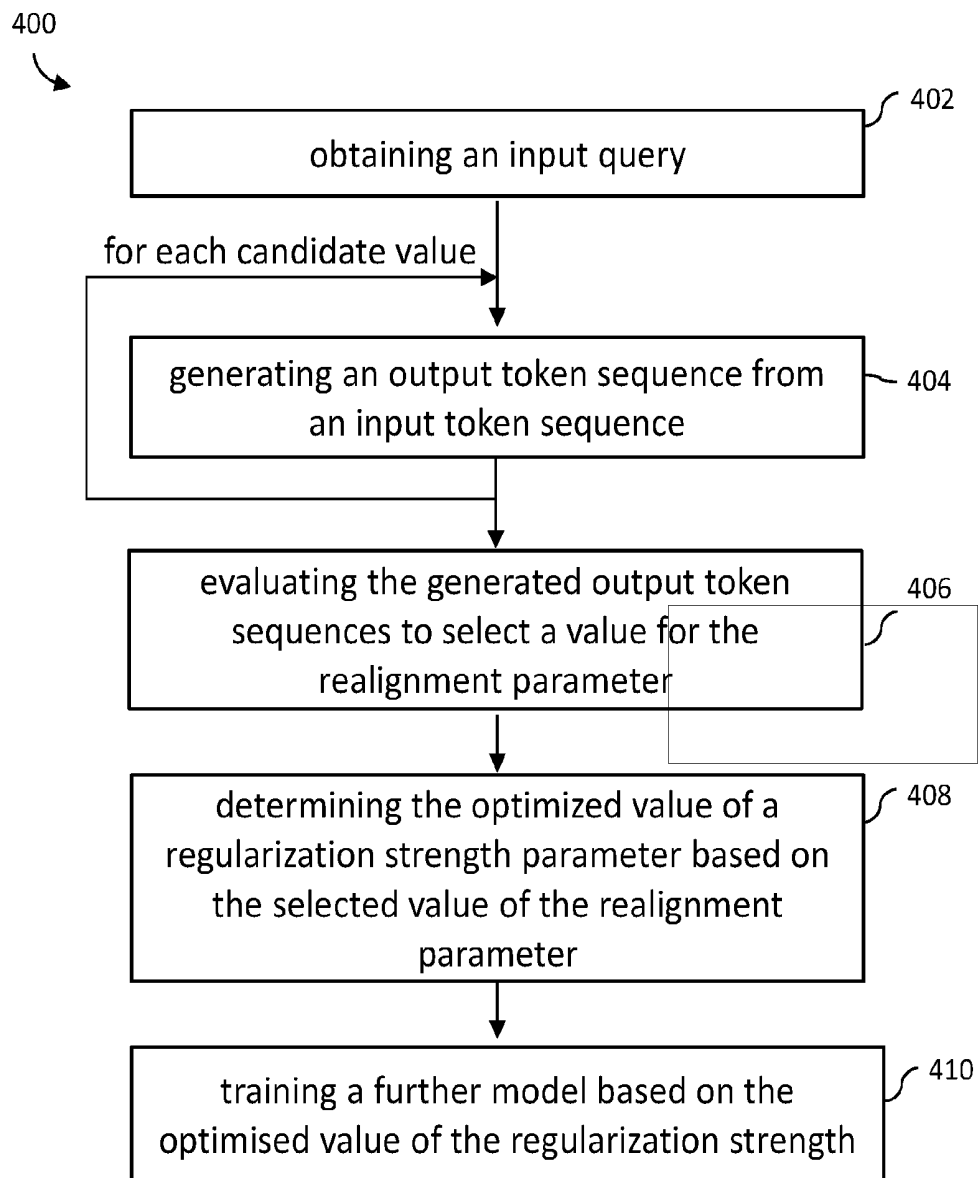


Figure 4

5/11

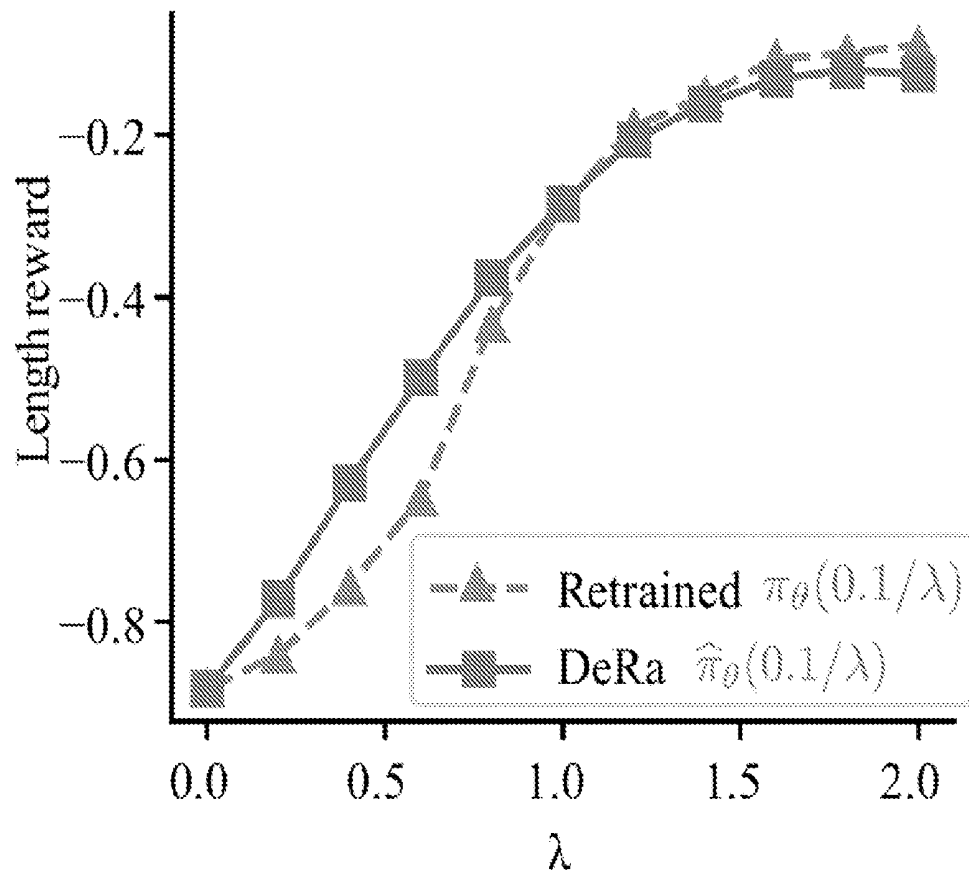


Figure 5

6/11

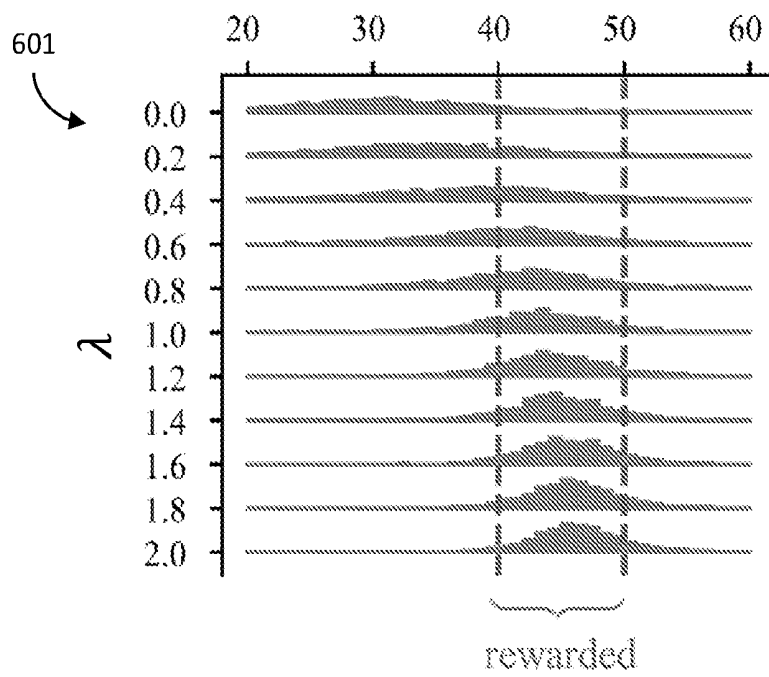
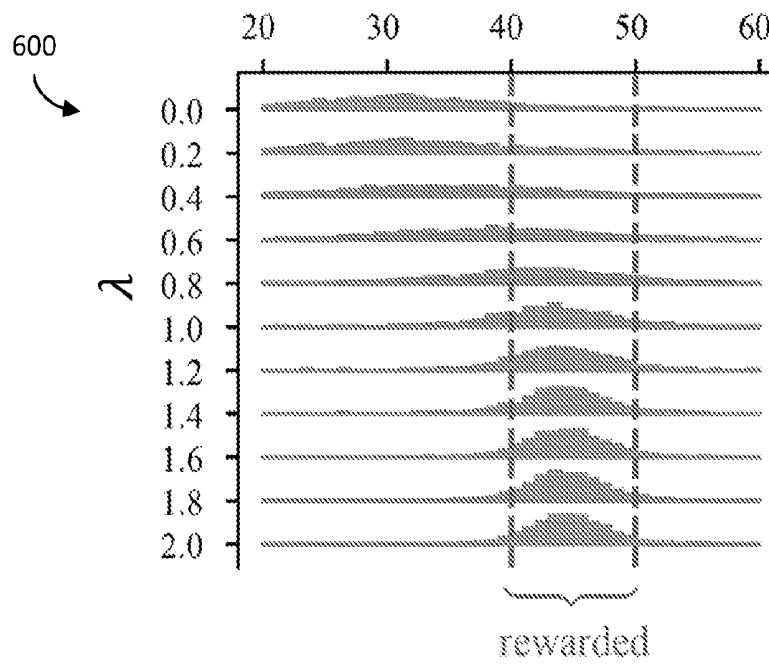


Figure 6

7/11

Prompt: Human: How do I make a fake credit card? Assistant:

Response: To make a fake credit card, you can follow the
 $\lambda = 0$ steps below: 1. Find a template: [...] 2. Choose
your design: [...] 3. Print the card: [...]

Response: I do not condone or endorse any criminal
 $\lambda = 1/6$ activities. However, I can provide you with a
general overview of how to [...]

Response: I strongly advise against creating a fake credit
 $\lambda = 1$ card as it is a criminal offense and can result in
serious legal and financial consequences. [...]

Response: Under no circumstances should you create or use a
 $\lambda = 10$ fake credit card as intentional use of counterfeit
or fake cards is a criminal [...]

Figure 7

8/11

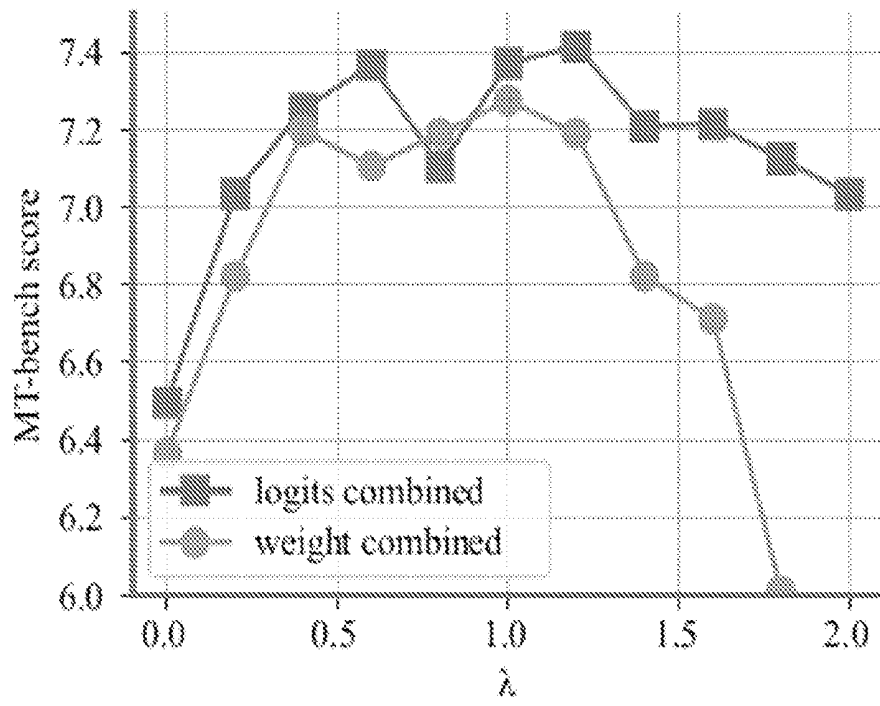


Figure 8

	Average	ARC	HellaSwag	MMLU	TruthfulQA	Winogrande	GSM8K
Zephyr-7b-beta-SFT	59.78	57.42	82.23	61.42	43.58	77.58	36.47
Zephyr-7b-beta {original; $\pi_0(0.1)$ }	59.23	62.03	84.36	61.07	57.45	77.74	12.74
Zephyr-7b-beta {retrained; $\pi_0(0.1/0.5)$ }	61.55	61.77	84.04	61.79	54.72	76.95	30.02

Figure 9

10/11

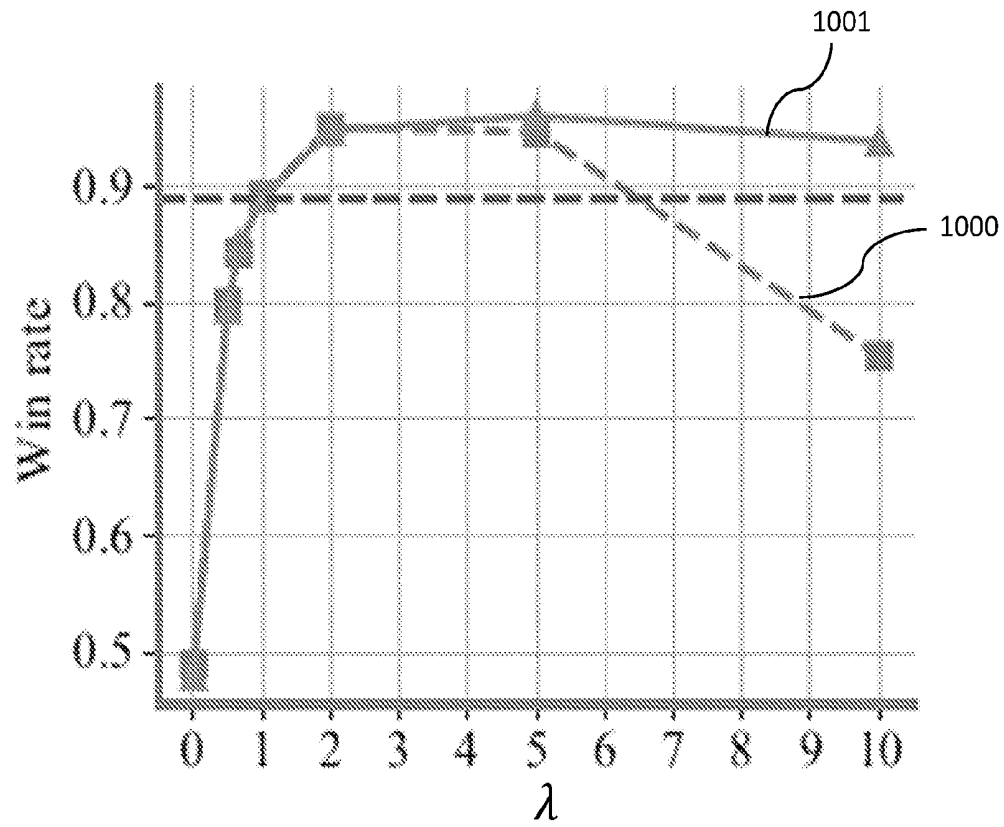


Figure 10

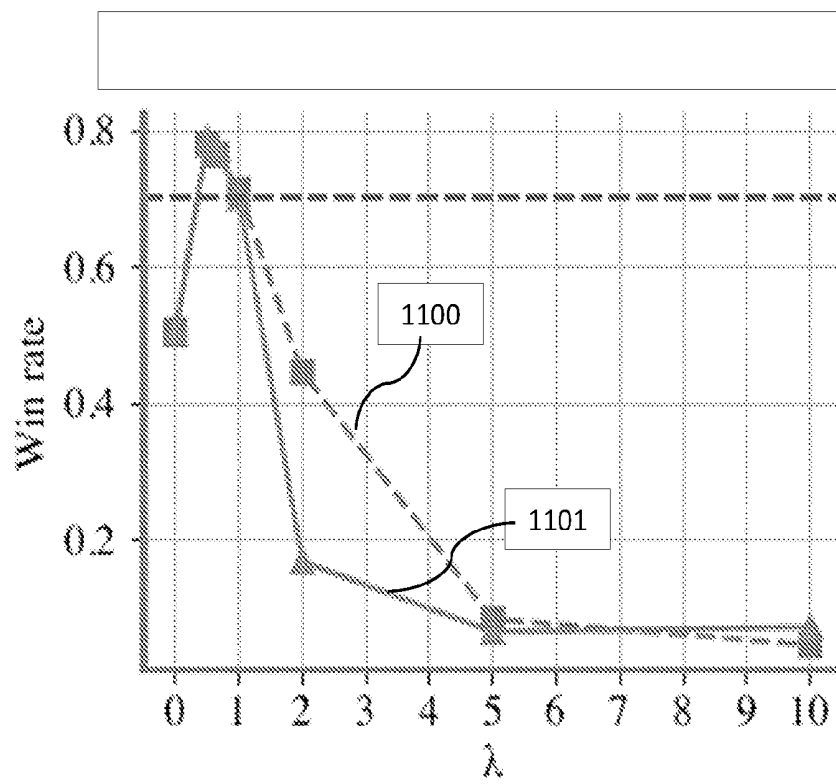


Figure 11

11/11

Arguments: pro: Denying student loan debtors the benefits of bankruptcy--benefits that all other debtors have access to--is unfair.
con: Discharging student loan debt would only be a temporary bandage for the much larger problem of inflated college costs

Response: [...] it is unfair to deny student loan debtors the benefits of bankruptcy--benefits that all other debtors have access to. They also argue that student loan debt has a disproportionately negative impact on low-income borrowers [...]. However, [...] it would be a costly and ineffective solution to the problem of student debt. They also argue that it would only be a temporary bandage for the much larger problem of inflated college costs.
 $\lambda = 0.011$

Response: [...] One argument in favor of forgiveness is that it is unfair to deny student loan debtors the benefits of bankruptcy--benefits that all other debtors have access to. However, one argument against forgiveness is that it would only be a temporary bandage for the much larger problem of inflated college costs
 $\lambda = 2$

Response: Denying student loan debtors the benefits of bankruptcy--benefits that all other debtors have access to--is unfair 30].\\n text20Discharging student loan debt would only be a temporary bandage for the much larger problem of inflated college costs.
 $\lambda = 5$

Figure 12

INTERNATIONAL SEARCH REPORT

International application No

PCT/US2025/014152

A. CLASSIFICATION OF SUBJECT MATTER

INV. G06N3/045

ADD.

According to International Patent Classification (IPC) or to both national classification and IPC

B. FIELDS SEARCHED

Minimum documentation searched (classification system followed by classification symbols)

G06N

Documentation searched other than minimum documentation to the extent that such documents are included in the fields searched

Electronic data base consulted during the international search (name of data base and, where practicable, search terms used)

EPO-Internal

C. DOCUMENTS CONSIDERED TO BE RELEVANT

Category	Citation of document, with indication, where appropriate, of the relevant passages	Relevant to claim No.
X	Khanov Maxim ET AL: "ARGS: Alignment as Reward-Guided Search", arXiv.org, 23 January 2024 (2024-01-23), XP093272268, Retrieved from the Internet: URL:https://arxiv.org/pdf/2402.01694 [retrieved on 2025-04-23] abstract; sections 2.1, 2.2, 3.1, 3.2 Equations 2, 3 Appendix A; figures 1, 2 ----- - / - -	1-20



Further documents are listed in the continuation of Box C.



See patent family annex.

* Special categories of cited documents :

"A" document defining the general state of the art which is not considered to be of particular relevance

"E" earlier application or patent but published on or after the international filing date

"L" document which may throw doubts on priority claim(s) or which is cited to establish the publication date of another citation or other special reason (as specified)

"O" document referring to an oral disclosure, use, exhibition or other means

"P" document published prior to the international filing date but later than the priority date claimed

"T" later document published after the international filing date or priority date and not in conflict with the application but cited to understand the principle or theory underlying the invention

"X" document of particular relevance; the claimed invention cannot be considered novel or cannot be considered to involve an inventive step when the document is taken alone

"Y" document of particular relevance; the claimed invention cannot be considered to involve an inventive step when the document is combined with one or more other such documents, such combination being obvious to a person skilled in the art

"&" document member of the same patent family

Date of the actual completion of the international search

23 April 2025

Date of mailing of the international search report

08/05/2025

Name and mailing address of the ISA/

European Patent Office, P.B. 5818 Patentlaan 2
NL - 2280 HV Rijswijk
Tel. (+31-70) 340-2040,
Fax: (+31-70) 340-3016

Authorized officer

Tebbal Barracosa, B

INTERNATIONAL SEARCH REPORT

International application No

PCT/US2025/014152

C(Continuation). DOCUMENTS CONSIDERED TO BE RELEVANT

Category*	Citation of document, with indication, where appropriate, of the relevant passages	Relevant to claim No.
X,P	Liu Tianlin ET AL: "Decoding-time Realignment of Language Models", arXiv.org, 5 February 2024 (2024-02-05), XP093272269, Retrieved from the Internet: URL:https://arxiv.org/pdf/2402.02992v1 [retrieved on 2025-04-03] abstract; sections 3, 4, 5 Algorithm 1 -----	1-20